Smart Multi-Sensor Wearable for Real-Time Elderly Health Anomaly and Fall Detection

INDIVIDUAL THESIS

Lakshan N K T

(IT22196156)

Individual component: Waist-Worn Smart Belt for Real-Time Fall Detection

Group ID: R26-IT-108

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) in Information Technology

Specialization: Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

Submission: October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Lakshan N K T

Student ID: IT22196156

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Nelum Chathuranga Amarasena		
Co-supervisor: Mr. Samadhi Rathnayake		

ABSTRACT

Falls are common in people with Parkinson's disease, and a person who falls may be unable to get up or call for help. This thesis presents the Smart Belt, the waist-worn fall detection component of the SmartGlove healthcare Internet of Things system. The belt combines an ESP32-S3 microcontroller with an MPU6050 accelerometer and gyroscope whose built-in 21 Hz low-pass filter removes noise without software delay. Every 100 ms the firmware computes the signal magnitude vector and the trunk tilt angle and applies a two-stage rule: an impact above 24.0 m/s² must be followed within 1.5 s by a tilt above 55°. A confirmed fall is latched for 15 s and sent immediately to a local hub, whose backend treats it as an emergency and sends an alert with the user's live location through Firebase to carers' mobile phones. The belt also classifies posture as Standing, Sitting, Walking, Running or Lying from the tilt angle and the variance of the acceleration magnitude, and shares it with the system as activity context. Thresholds were chosen instead of machine learning to give minimal latency, independence from the network, low power consumption and explainable alerts. The belt was evaluated with 120 trials, comprising 48 simulated falls (forward, backward, lateral, and fall-from-sitting) and 72 activities of daily living (walking, running, sitting down quickly, lying down, bending, and stair climbing) performed by healthy adult volunteers on a mattress, achieving a sensitivity of **95.8%** and a specificity of **97.2%**. End-to-end alerts reached a mobile phone in 23 of 24 tests (95.8%) with a mean delay of 1.8 s from impact to notification. The results indicate that a low-cost, explainable, on-device two-stage threshold algorithm can reliably distinguish falls from common daily activities and deliver emergency alerts through a local hub and cloud backend without depending on network availability for the detection decision. The individual contribution of this work is the design and implementation of the belt hardware and firmware, the fall detection and posture logic, the two-stage threshold algorithm, the telemetry protocol, and their integration with the system's emergency alert workflow.

Keywords: fall detection, wearable sensors, Parkinson's disease, accelerometer, edge computing

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Mr. Nelum Chathuranga Amarasena, and my co-supervisor, Mr. Samadhi Rathnayake, for their guidance and support throughout this project. I also thank the members of my project group, R26-IT-108, with whom the SmartGlove system was developed, and the volunteers who participated in the fall and activity trials for their time and cooperation, and my fellow group members who assisted with hardware assembly, testing setup and data collection during the evaluation of the Smart Belt. Finally, I thank the staff of the Sri Lanka Institute of Information Technology for providing the facilities used in this work.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 2
1.2.1 Falls in people with Parkinson's disease 2
1.2.2 Wearable fall detection and waist-worn sensing 3
1.2.3 The Smart Belt component 3
1.3 Problem statement 4
1.4 Research gap 5
1.5 Aim and objectives 6
1.5.1 Main objective 6
1.5.2 Specific objectives 6
1.6 Research questions or hypotheses 7
1.7 Scope limitations and assumptions 7
1.7.1 Scope 7
1.7.2 Limitations 7
1.7.3 Assumptions 7
1.8 Individual contribution 8
1.9 Thesis structure 8
2 LITERATURE REVIEW AND RELATED WORK 9
2.1 Review method and scope 9
2.2 Domain and theoretical foundations ..... **Error! Bookmark not defined.**
2.2.1 Falls and fall risk in Parkinson's disease 9
2.2.2 Inertial sensing with MEMS accelerometers and gyroscopes 10
2.2.3 Signal magnitude vector and tilt estimation 10

2.2.4 Threshold-based, machine learning and deep learning approaches 11
2.2.5 Edge computing for wearable health monitoring 12
2.3 Related research 13
2.3.1 Wearable fall detection systems 13
2.3.2 Sensor placement: waist versus wrist 14
2.3.3 Fall detection in people with Parkinson's disease 14
2.3.4 Posture and activity recognition 14
2.3.5 Discriminating falls from activities of daily living 15
2.4 Existing systems products and techniques 16
2.4.1 Commercial fall detection devices 16
2.4.2 Public fall detection datasets 16
2.5 Gap synthesis and conceptual framework 17
3 RESEARCH METHODOLOGY 18
3.1 Research design 18
3.2 Data participants or study materials 19
3.3 Experimental or study procedure 19
3.4 Measures and evaluation metrics 20
3.5 Data analysis 20
3.6 Ethics privacy security and responsible AI 21
3.6.1 Participant consent and safety 21
3.6.2 Privacy of health and location data 21
3.6.3 Security of device communication 21
3.6.4 Explainability and responsible use 22
3.7 Reproducibility and research artefacts 22
4 PRODUCT AND SOLUTION DESIGN 23
4.1 Requirements 23

4.1.1 Functional requirements 23
4.1.2 Non-functional requirements 24
4.2 Component architecture within the complete solution 24
4.3 Design alternatives and decisions 25
4.3.1 Threshold-based detection versus on-device machine learning 26
4.3.2 Hardware digital low-pass filter versus software Butterworth filter 26
4.3.3 Waist versus wrist placement 26
4.3.4 Sampling and transmission rates 26
4.3.5 Local hub versus direct cloud communication 27
4.4 Detailed design 27
4.4.1 Hardware design 27
4.4.2 Feature computation 28
4.4.3 Fall detection logic 28
4.4.4 Posture classification logic 30
4.4.5 Telemetry message design 31
4.5 Standards and professional practice 32
5 IMPLEMENTATION INTEGRATION AND TESTING 33
5.1 Development environment and technology stack 33
5.2 Component implementation 33
5.2.1 Sensor initialisation and configuration 33
5.2.2 Sampling loop and circular buffer 34
5.2.3 Signal magnitude vector and tilt computation 34
5.2.4 Fall detection and alert latching 34
5.2.5 Posture classification 35
5.2.6 OLED status display 35
5.2.7 Network connection and hub discovery 36

5.3 Integration with the other components 36
5.3.1 Telemetry transmission to the hub 36
5.3.2 Backend handling of fall events 37
5.3.3 Posture as activity context 38
5.4 Verification and software testing 38
5.4.1 Hardware and sensor verification 38
5.4.2 Functional test cases 39
5.4.3 Integration testing 39
5.5 Deployment and operation 40
6 RESULTS AND VALIDATION 41
6.1 Research results 41
6.2 Product and technical results 41
6.3 User or stakeholder validation 42
6.4 Integration results 42
6.5 Summary of findings 43
7 DISCUSSION 44
7.1 Interpretation of findings 44
7.2 Comparison with literature baselines and existing solutions 45
7.3 Research contribution 45
7.4 Product contribution 46
7.5 Limitations and threats to validity 46
7.5.1 Internal and construct validity 46
7.5.2 External validity 46
7.5.3 Technical limitations 46
7.6 Implications 47
8 COMMERCIALISATION AND SUSTAINABILITY 48

8.1 Value proposition and target users 48
8.2 Market and competitive position 48
8.3 Business or sustainability model 49
8.4 Scalability intellectual property and compliance 49
8.5 Contribution to the Sustainable Development Goals 50
9 CONCLUSION AND FUTURE DIRECTIONS 51
9.1 Conclusion 51
9.2 Achievement of objectives 51
9.3 Individual research and product contributions 51
9.4 Future directions 52
REFERENCES 53
GLOSSARY 55
APPENDICES 56
Appendix A Smart Belt firmware listing 56
Appendix B Academic integrity and AI use disclosure 64

LIST OF FIGURES

Figure 1.1: Overall SmartGlove system with the Smart Belt component highlighted 2

Figure 1.2: Photograph of the Smart Belt prototype (Original in Color) 4

Figure 2.1: Accelerometer axes and the tilt angle relative to gravity 11

Figure 2.2: Typical phases of a fall 12

Figure 2.3: Conceptual framework of the Smart Belt fall detection component 17

Figure 3.1: Research process followed for the Smart Belt component 18

Figure 4.1: Architecture of the complete solution showing the Smart Belt and its interfaces 25

Figure 4.2: Connection diagram of the Smart Belt 28

Figure 4.3: Mounting position and sensor axis orientation on the body 28

Figure 4.4: Fall detection flowchart 29

Figure 4.5: State diagram of the fall detection logic 30

Figure 4.6: Posture classification decision tree 31

Figure 5.1: Sensor initialisation and configuration (excerpt from belt.ino) 34

Figure 5.2: Non-blocking 10 Hz sampling and circular buffer (excerpt from belt.ino) 34

Figure 5.3: Feature computation (excerpt from belt.ino, simplified variable names) 34

Figure 5.4: Fall detection and latching (excerpt from belt.ino) 35

Figure 5.5: Posture classification (excerpt from belt.ino) 35

Figure 5.6: OLED normal dashboard and emergency fall screen (Original in Color) 36

Figure 5.7: Regular and immediate telemetry transmission (excerpt from belt.ino) . 37

Figure 5.8: Handling of a fall event in the backend (excerpt from backend/main.py) 37

Figure 5.9: Sequence of the fall alert from the belt to the mobile phone 38

Figure 5.10: Smart Belt worn on the waist (Original in Color) 40

LIST OF TABLES

Table 1.1: Comparison of selected existing studies and the proposed Smart Belt component 5

Table 2.1: Search terms, sources and inclusion criteria 9

Table 2.2: Summary of related fall detection studies 15

Table 2.3: Comparison of existing fall detection products and techniques 16

Table 3.1: Hardware components of the Smart Belt 19

Table 3.2: Fall and activity-of-daily-living scenarios tested 19

Table 3.3: Evaluation metrics and their definitions 20

Table 3.4: Firmware configuration parameters 22

Table 4.1: Functional requirements of the Smart Belt 23

Table 4.2: Non-functional requirements 24

Table 4.3: Summary of design alternatives and the decisions taken 27

Table 4.4: Fields of the belt telemetry message 31

Table 5.1: Development tools, libraries and versions 33

Table 5.2: Functional test cases and outcomes 39

Table 6.1: Confusion matrix of fall detection results 41

Table 6.2: Fall detection performance 41

Table 6.3: Results by scenario 41

Table 6.4: Detection latency and false alarm rate 42

Table 6.5: Posture classification results 42

Table 6.6: End-to-end fall alert delivery results 42

Table 7.1: Comparison of the Smart Belt with related work 45

Table 8.1: Competitive comparison with existing fall detection products 48

Table 9.1: Achievement of each specific objective 51

Table B.1: Use of generative AI tools..... 64

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ADL	Activity of Daily Living
DLPF	Digital Low-Pass Filter
FN	False Negative
FP	False Positive
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
I2C	Inter-Integrated Circuit
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
JSON	JavaScript Object Notation
MEMS	Micro-Electro-Mechanical Systems
OLED	Organic Light-Emitting Diode
PD	Parkinson's Disease
RQ	Research Question
SCL	Serial Clock Line
SDA	Serial Data Line
SDG	Sustainable Development Goal
SMV	Signal Magnitude Vector
TN	True Negative
TP	True Positive
UDP	User Datagram Protocol

1 INTRODUCTION

This chapter introduces the Smart Belt, a waist-worn wearable that detects falls on the device and reports posture to the wider SmartGlove healthcare Internet of Things (IoT) system. It describes the overall project, the context of the individual component, the problem addressed, the research gap, the aim and objectives, the research questions, the scope of the work and the structure of the thesis.

1.1 Overall project context

The group project developed a wearable healthcare IoT system for people with Parkinson's disease (PD). The system consists of a smart glove worn on the wrist and hand, a waist-worn Smart Belt, a local hub that runs a Python backend and a live dashboard, a Firebase cloud database, and mobile phones used by carers and nurses to receive emergency notifications. Each wearable sends its data to the local hub rather than directly to the internet, which reduces power consumption on the wearables and allows monitoring to continue on the local network. The hub combines the data from both wearables, and an emergency notification that includes the user's live Global Positioning System (GPS) location is sent through Firebase to mobile phones when an emergency condition occurs. Figure 1.1 shows the complete system and the position of the Smart Belt within it. The glove and its health screening functions were developed by other members of the group and are outside the scope of this thesis.

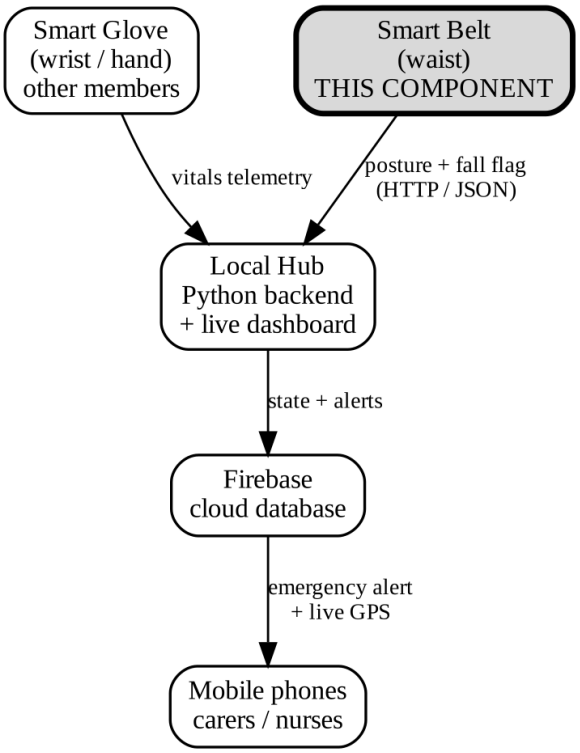


Figure 1.1: Overall SmartGlove system with the Smart Belt component highlighted

1.2 Individual component context

1.2.1 Falls in people with Parkinson's disease

Falls are a frequent and serious problem for people with PD. In a prospective study, Bloem et al. [1] followed 59 people with PD and 55 healthy controls for six months and found that 50.8% of the patients fell at least once, compared with 14.5% of the controls, and that 25.4% of the patients fell repeatedly. Most of the falls in the patient group had intrinsic, patient-related causes, and 62% of them resulted in soft-tissue injury. A later meta-analysis of six prospective studies, covering 473 people with PD, estimated that 46% of patients fell within a three-month period and that 21% of those with no previous fall history still fell during that period [2]. These figures show that falls in PD are common, recurrent and difficult to predict from clinical history alone.

The consequences of a fall depend not only on the impact but also on how quickly help arrives. Fleming and Brayne [3] showed in a cohort of people aged over 90 that inability to get up after a fall and long periods lying on the floor were common, were associated with serious injury, hospital admission and moving into long-term care, and that many people who owned a call alarm did not use it to summon help. A person

with PD who falls may face the same difficulty, particularly when motor symptoms limit the ability to stand up or reach a telephone. This motivates detection that does not depend on the person raising the alarm.

1.2.2 Wearable fall detection and waist-worn sensing

Automatic fall detectors use sensors either in the environment, such as cameras and floor sensors, or on the body [4]. Wearable detectors based on inertial sensors are inexpensive, work indoors and outdoors and do not record images of the user, which makes them attractive for continuous use at home [4], [5]. A fall typically produces a short period of large acceleration at impact, followed by a change in body orientation as the person comes to rest on the ground [5]. A tri-axial accelerometer worn on the trunk captures both effects, and several studies have reported that the waist is one of the most effective and practical locations for a single fall detection sensor because it is close to the body's centre of mass [6], [7].

1.2.3 The Smart Belt component

The Smart Belt is the individual component described in this thesis. It is built around an ESP32-S3 SuperMini microcontroller board [8], an MPU6050 accelerometer and gyroscope module [9] and a small SSD1306 Organic Light-Emitting Diode (OLED) display. The firmware samples the accelerometer every 100 ms, computes the Signal Magnitude Vector (SMV) and a tilt angle, and applies a two-stage rule to detect a fall: an impact, followed within 1.5 s by a large tilt of the trunk. A detected fall is held active for 15 s so that it is reliably delivered to the hub. The belt also classifies the wearer's posture as Standing, Sitting, Walking, Running or Lying, and sends this posture state to the hub, where it is used as activity context by the rest of the system. Figure 1.2 shows the prototype.



Figure 1.2: Photograph of the Smart Belt prototype (Original in Color)

1.3 Problem statement

People with PD fall frequently and may be unable to get up or call for help after a fall, which increases the risk of injury and of long periods on the floor [1], [3]. Manual alarm buttons depend on the person being conscious, able and willing to press them. Many automatic detectors process sensor data on a smartphone or in the cloud, which adds delay and fails when the network is unavailable, while detectors that rely on complex machine learning models are difficult to run on very small, low-power devices and difficult for carers and clinicians to interpret [4], [10]. In addition, a fall detector that works in isolation cannot share the wearer's posture and activity with other sensors that would benefit from that context.

The problem addressed in this thesis is therefore how to detect falls of a person with PD automatically, quickly and explainably on a low-cost waist-worn device, and how to deliver each detected fall, together with the wearer's posture, to a wider monitoring system that can raise an emergency alert.

1.4 Research gap

Research on wearable fall detection is extensive, but several gaps remain that are relevant to this component. First, most algorithms have been developed and evaluated with simulated falls performed by young volunteers, and their performance drops when they are applied to real-world falls [11], [4]. Second, relatively few studies target people with PD specifically, even though this population falls more often than the general older population [1], [12]. Third, many high-accuracy approaches rely on machine learning classifiers trained offline and executed on a smartphone or a computer [13], which makes them dependent on that device and less transparent to clinical users. Finally, published detectors are usually evaluated as stand-alone devices; few are integrated into a multi-sensor monitoring system in which the detector's posture output is reused as context and its fall output triggers an emergency workflow that includes the user's location.

Table 1.1 compares selected existing work with the proposed component on these characteristics.

Table 1.1: Comparison of selected existing studies and the proposed Smart Belt component

Work	Placement	Processing	Method	PD focus	Integrated alert and posture context
Bourke et al. [14]	Trunk and thigh	Offline analysis	Threshold	No	No
Kangas et al. [6]	Waist, wrist, head	Offline analysis	Threshold and posture	No	No
Karantonis et al. [15]	Waist	Real time, on device	Rule-based classifier	No	Partial (posture)
Özdemir and Barshan [13]	Six body locations	Offline	Machine learning	No	No
Silva de Lima et al. [12]	Neck-worn alarm pendant	Commercial device	Proprietary	Yes	Alert only

Work	Placement	Processing	Method	PD focus	Integrated alert and posture context
Proposed Smart Belt	Waist	Real time, on device	Two-stage threshold	Yes (target users)	Yes

In summary, there is a need for an explainable, on-device fall detector designed for people with PD that is integrated into a wider monitoring system, sharing posture context and triggering location-aware emergency alerts.

1.5 Aim and objectives

1.5.1 Main objective

The main objective of this work was to design, implement and evaluate a waist-worn Smart Belt that detects falls in real time on the device and delivers fall events and posture context to the SmartGlove system for emergency alerting.

1.5.2 Specific objectives

The specific objectives were:

1. To build a waist-worn sensing unit based on the MPU6050 inertial sensor and the ESP32-S3 microcontroller.

2. To implement on-device feature extraction, namely the signal magnitude vector, the trunk tilt angle and the variance of the acceleration magnitude over a sliding window.

3. To implement a two-stage threshold fall detection algorithm that confirms an impact with a subsequent change in trunk orientation and latches the fall alert.

4. To implement a rule-based posture and activity classifier that reports Standing, Sitting, Walking, Running and Lying states.

5. To integrate the belt with the local hub and backend so that a detected fall triggers an emergency alert and the posture state is available to the rest of the system.

6. To evaluate the fall detection performance of the belt and the delivery of fall alerts through the system.

1.6 Research questions or hypotheses

The work was guided by the following research questions, derived from the specific objectives:

RQ1. Can a two-stage impact-and-tilt threshold algorithm running on an ESP32 microcontroller distinguish falls from common activities of daily living (ADL)?

RQ2. How quickly and reliably can a fall detected on the belt be delivered as an emergency alert through the hub and the cloud?

RQ3. Can the tilt angle and the variance of the acceleration magnitude measured at the waist provide a usable posture and activity state for the wider system?

1.7 Scope limitations and assumptions

1.7.1 Scope

The scope of this thesis covers the hardware of the Smart Belt, its firmware, the fall detection and posture classification logic, the transmission of telemetry to the local hub, and the way the backend handles a fall event. The smart glove, the health screening functions of the hub and the mobile application are owned by other members of the group and are described only where they interact with the belt.

1.7.2 Limitations

The main limitations, discussed in detail in Section 7.5, are that the belt samples the accelerometer at 10 Hz, that the tilt angle is computed from a single sensor axis and therefore depends on how the belt is worn, that the firmware confirms a fall by trunk tilt rather than by a period of inactivity after the impact, that the evaluation was carried out under controlled conditions using simulated falls performed by healthy adult volunteers onto a mattress rather than real falls by people with Parkinson's disease in daily life, and that the prototype's network communication was not hardened for deployment.

1.7.3 Assumptions

It was assumed that the belt is worn at the waist in a fixed position and orientation, that a Wi-Fi network is available to connect the belt to the hub, and that the hub, the backend and Firebase are running when an alert must be delivered.

1.8 Individual contribution

My individual contribution to the SmartGlove group project was the design, implementation and evaluation of the waist-worn Smart Belt component. This included the belt hardware (ESP32-S3 SuperMini, MPU6050 and SSD1306 OLED), the complete firmware (belt.ino) covering sensor sampling, feature extraction, two-stage fall detection with latching, posture classification, OLED display, hub discovery and telemetry transmission, the fall handling in the backend (backend/main.py), and the verification and testing of the belt, including bench tests, simulated fall and ADL trials, and end-to-end alert tests. The smart glove, the hub's health screening functions, the mobile application and the overall system architecture were developed by the other members of group R26-IT-108.

1.9 Thesis structure

Chapter 2 reviews the literature on falls in PD, inertial sensing, fall detection methods, sensor placement, posture recognition and existing products, and synthesises the research gap. Chapter 3 describes the research methodology, the evaluation metrics and the ethical, privacy and security considerations. Chapter 4 presents the requirements, the architecture of the belt within the complete solution, the design alternatives and the detailed design. Chapter 5 describes the implementation, the integration with the hub and backend, and the verification activities. Chapter 6 presents the results. Chapter 7 discusses the findings, the contributions and the limitations. Chapter 8 considers commercialisation and sustainability, and Chapter 9 concludes the thesis and outlines future work.

2 LITERATURE REVIEW AND RELATED WORK

This chapter reviews the literature that informed the design of the Smart Belt. It first describes how the review was carried out, then presents the theoretical foundations of inertial fall detection, reviews related research and existing products, and finally synthesises the research gap into a conceptual framework.

2.1 Review method and scope

A narrative review was carried out using IEEE Xplore, PubMed, ScienceDirect, MDPI and Google Scholar. The search combined terms for the event (fall, fall detection), the technology (accelerometer, inertial sensor, wearable, waist, threshold, machine learning, edge computing) and the population (Parkinson's disease, older adults). Peer-reviewed journal and conference papers, device datasheets and standards were included when they addressed wearable inertial fall detection, sensor placement, posture recognition or falls in PD. Studies of camera-based or ambient-only systems were used only for context. Table 2.1 summarises the search strategy.

Table 2.1: Search terms, sources and inclusion criteria

Aspect	Description
Sources	IEEE Xplore, PubMed, ScienceDirect, MDPI, Google Scholar, manufacturer documentation
Search terms	"fall detection" AND (accelerometer OR wearable OR waist OR threshold OR "machine learning"); "falls" AND "Parkinson's disease"; "edge computing" AND IoT
Included	Peer-reviewed studies on wearable inertial fall detection, sensor placement, posture and activity recognition, falls in PD; datasheets and standards
Excluded	Vision-only and ambient-only systems (used for context only); non-peer-reviewed claims

2.2 Domain and theoretical foundations

2.2.1 Falls and fall risk in Parkinson's disease

PD is a progressive neurological disorder whose motor symptoms include bradykinesia, rigidity, tremor and postural instability. Postural instability and gait problems make falls one of the most common complications of the disease. As described in Section 1.2.1, around half of people with PD fall within six months [1] and almost half within three months [2], and a quarter fall repeatedly [1]. Pickering et al. [2] found that previous falls were the strongest predictor of future falls, but that a

fifth of patients without a fall history still fell, which means that monitoring cannot be restricted to known fallers. These findings justify continuous, automatic fall detection for this population.

2.2.2 Inertial sensing with MEMS accelerometers and gyroscopes

Micro-Electro-Mechanical Systems (MEMS) accelerometers measure the specific force acting on a small proof mass along three orthogonal axes. When the sensor is stationary the measured vector equals gravity, approximately 9.81 m/s², so the accelerometer reports both the orientation of the body segment and its dynamic acceleration [16]. MEMS gyroscopes measure angular velocity about the same axes. Mathie et al. [16] described accelerometry as an integrated and practical method for long-term ambulatory monitoring of human movement, able to identify postures, movements and falls with a single body-worn device.

The MPU6050 used in this work combines a three-axis accelerometer and a three-axis gyroscope in one package with an Inter-Integrated Circuit (I2C) interface [9]. Its accelerometer range is selectable from ±2 g to ±16 g and its gyroscope range from ±250 °/s to ±2000 °/s. The device also contains a configurable Digital Low-Pass Filter (DLPF) that limits the bandwidth of both sensors before the data are read by the host [9]. These features allow filtering to be performed in hardware rather than in software.

2.2.3 Signal magnitude vector and tilt estimation

Two features derived from the three accelerometer axes are widely used in fall detection [14], [6]. The signal magnitude vector, also called the resultant or total acceleration, combines the three axes so that the result does not depend on the orientation of the sensor:

$$SMV = \sqrt{(ax^2 + ay^2 + az^2)} \quad (2.1)$$

At rest the SMV is close to 9.81 m/s². During an impact it rises sharply, and during the free-fall phase before an impact it may fall well below 1 g. Because the SMV is orientation independent, a single threshold on the SMV can detect impacts regardless of the direction of the fall [14].

The orientation of the trunk can be estimated from the direction of gravity in the sensor frame. When the body is quasi-static, the angle between one sensor axis and the horizontal plane is given by

$$\theta = \operatorname{atan2}(a_x, \sqrt{(a_y^2 + a_z^2)}) \quad (2.2)$$

where θ is 0° when the chosen axis is horizontal and ±90° when it is aligned with gravity. Figure 2.1 illustrates these quantities. A change from an upright to a horizontal trunk after an impact is a strong indicator that a person has fallen and is lying on the ground [6], [15].

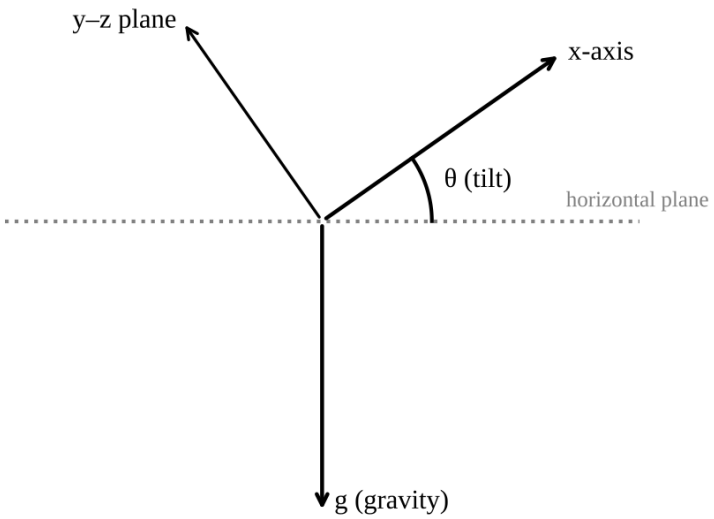


Figure 2.1: Accelerometer axes and the tilt angle relative to gravity

2.2.4 Threshold-based, machine learning and deep learning approaches

Noury et al. [5] divided a fall into a pre-fall phase, a critical phase containing the free fall and the impact, a post-fall phase in which the person usually lies still, and a recovery phase, as shown in Figure 2.2. Detection methods differ in which of these phases they use and how they make the decision.

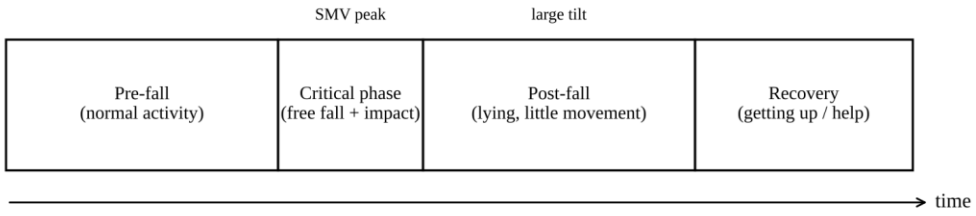


Figure 2.2: Typical phases of a fall

Threshold-based methods compare one or more features with fixed limits. They are simple, require very little computation and memory, and their decisions can be explained directly in terms of physical quantities. Bourke et al. [14] showed that a single upper threshold on the trunk resultant acceleration could separate simulated falls from ADL in their dataset, and Kangas et al. [6] showed that combining an impact threshold with a posture check after the impact reduced false alarms. The weakness of thresholds is that fixed limits may not generalise across people, activities and real-world conditions [11], [4].

Machine learning methods extract a set of features from windows of sensor data and train a classifier such as a support vector machine, k-nearest neighbours or a random forest. Özdemir and Barshan [13] compared several classifiers using six body-worn sensor units and reported accuracies above 99% for the best classifiers on simulated falls. Deep learning methods learn features directly from raw signals with convolutional or recurrent neural networks. Both approaches can model complex patterns, but they require large labelled datasets, are harder to run on small microcontrollers and are less transparent to users and clinicians [4].

2.2.5 Edge computing for wearable health monitoring

Edge computing places computation close to where data are produced instead of in a distant cloud data centre. Shi et al. [10] identified reduced response time, lower bandwidth use, better energy efficiency and improved privacy as the main benefits. For fall detection these benefits are directly relevant: a decision made on the wearable is not delayed by the network, continues to work when the connection is lost, and only a small event message needs to be transmitted. Modern microcontrollers such as the ESP32-S3, which integrates a dual-core processor with Wi-Fi and Bluetooth Low

Energy [8], make it practical to perform feature extraction and decision making on the wearable itself.

2.3 Related research

2.3.1 Wearable fall detection systems

In 2007, the study of "Evaluation of a threshold-based tri-axial accelerometer fall detection algorithm" [14] proposed detecting falls by applying an upper threshold to the resultant acceleration measured at the trunk and evaluated it on simulated falls by young subjects and ADL performed by older adults, and achieved separation of falls from ADL across a dataset of 480 movements using a trunk-mounted sensor. The study established the SMV impact threshold as a simple and effective first stage for fall detection.

In 2008, the study of "Comparison of low-complexity fall detection algorithms for body attached accelerometers" [6] proposed and compared algorithms that combined start-of-fall, impact and posture-after-fall conditions using accelerometers worn at the waist, head and wrist, and achieved the best results with sensors at the waist and head, with the posture check after the impact contributing to high specificity. This two-stage principle of impact followed by posture confirmation is the basis of the algorithm implemented in the Smart Belt.

In 2006, the study of "Implementation of a real-time human movement classifier using a triaxial accelerometer for ambulatory monitoring" [15] proposed a waist-mounted device that classified activity, posture and falls in real time on the device using a hierarchical set of rules, and achieved accurate classification of postures and movements in laboratory trials. The study demonstrated that waist accelerometry and simple rules can run continuously on embedded hardware.

In 2014, the study of "Detecting falls with wearable sensors using machine learning techniques" [13] proposed applying six machine learning classifiers to features from six wearable sensor units and achieved accuracies above 99% for the best classifiers on simulated falls and ADL. The study showed the potential of machine learning, but its processing was performed offline on a computer using data from several sensors.

2.3.2 Sensor placement: waist versus wrist

Sensor placement has a strong influence on fall detection performance. Kangas et al. [6] found that the waist and the head gave better results than the wrist, because wrist movements during ADL are large and varied and are only loosely coupled to the movement of the body's centre of mass. In 2016, the study of "An analysis on sensor locations of the human body for wearable fall detection devices: Principles and practice" [7] proposed a systematic comparison of single sensor locations and combinations, and achieved the highest single-location performance with the waist. Wrist-worn detectors such as smartwatches are more acceptable to many users, but they must compensate for the larger variability of arm movements [4]. These results support placing the fall detector of the SmartGlove system on a belt rather than in the glove.

2.3.3 Fall detection in people with Parkinson's disease

People with PD present specific challenges for fall detection, including tremor, slowed movements, freezing of gait and frequent near-falls. In 2020, the study of "Home-based monitoring of falls using wearable sensors in Parkinson's disease" [12] proposed using data from a commercially available wearable personal emergency response device with automatic fall detection to monitor falls of people with PD in daily life, and achieved a large-scale, home-based record of detected falls in this population. The study shows that wearable fall detection is feasible and useful for people with PD at home.

The device in that study was a necklace-worn personal emergency response pendant containing a tri-axial accelerometer and a barometer. Data from 2,063 people with PD and 2,063 matched controls, followed for about 1.1 years on average, showed 2.1 falls per person-year in the PD group compared with 0.7 in the controls, with 48% of the PD group falling at least once within a year [12].

2.3.4 Posture and activity recognition

The same waist-worn accelerometer used for fall detection can also recognise posture and activity. Mathie et al. [16] and Karantonis et al. [15] used the static component of acceleration to estimate trunk inclination, distinguishing upright from lying postures, and the dynamic component, measured by the variability of the acceleration signal, to

distinguish rest from walking and more vigorous activity. The Smart Belt follows this approach, using the tilt angle for posture and the variance of the SMV over a two-second window for activity.

2.3.5 Discriminating falls from activities of daily living

The main challenge for any fall detector is avoiding false alarms from ADL that produce large accelerations or large changes in orientation, such as sitting down heavily, jumping, running or lying down on a bed. Public datasets have been created so that algorithms can be tested on many types of falls and ADL. In 2017, the study of "SisFall: A fall and movement dataset" [17] proposed a dataset recorded with a waist-mounted device from young and older participants, covering many types of falls and ADL, and achieved a common benchmark for comparing threshold-based features. Casilari et al. [18] analysed the public datasets for wearable fall detection and found large differences in sensor placement, sampling rate, participants and fall types, which makes direct comparison of published results difficult. Bagalà et al. [11] evaluated thirteen published accelerometer-based algorithms on real-world falls and found that their performance was considerably lower than reported on simulated falls, mainly because of false alarms. Table 2.2 summarises the studies reviewed.

Table 2.2: Summary of related fall detection studies

Study	Sensor and placement	Method	Data	Main finding
[14]	Accelerometer, trunk and thigh	Upper threshold on SMV	Simulated falls, ADL	Trunk SMV threshold separated falls from ADL
[6]	Accelerometer, waist, head, wrist	Impact + posture thresholds	Simulated falls, ADL	Waist and head best; posture check raised specificity
[15]	Accelerometer, waist	Real-time rule-based classifier	Laboratory trials	Posture, activity and falls on device
[13]	Six inertial units	Six ML classifiers	Simulated falls, ADL	Best classifiers above 99% accuracy
[7]	Single and combined locations	ML classifiers	Simulated falls, ADL	Waist best single location
[17]	Accelerometers and gyroscope, waist	Threshold features	Public dataset	Benchmark of falls and ADL
[11]	Accelerometer, trunk	13 published algorithms	Real-world falls	Lower performance than on simulated falls
[12]	Neck-worn alarm pendant	Commercial detector	Home use, PD	Feasible home fall monitoring in PD

2.4 Existing systems products and techniques

2.4.1 Commercial fall detection devices

Commercial fall detection is available mainly in two forms. Personal emergency response systems provide a pendant or wrist button connected to a monitoring service, and some models add automatic fall detection; such a device was used for home monitoring of people with PD by Silva de Lima et al. [12]. Smartwatches also offer fall detection: the Apple Watch, for example, can detect a hard fall, alert the wearer and, if the wearer does not respond, call emergency services and notify emergency contacts [19]. These products are convenient, but they are closed systems whose algorithms cannot be inspected or adapted, they are worn on the wrist or neck rather than at the waist, and they do not share posture or fall information with other medical sensors worn by the same person.

2.4.2 Public fall detection datasets

Public datasets such as SisFall [17] allow fall detection algorithms to be developed and compared without new data collection. Casilari et al. [18] reviewed these datasets and noted that most contain simulated falls by young volunteers, that few include older adults and none of those reviewed focused on PD, and that sampling rates and sensor placement vary widely. These observations are relevant to the evaluation of the Smart Belt, because thresholds chosen on one dataset may not transfer to a device with a different sampling rate and placement. Table 2.3 compares the main existing approaches with the proposed component.

Table 2.3: Comparison of existing fall detection products and techniques

Approach	Placement	Where decision is made	Open / explainable	Shares posture with other sensors
Smartwatch fall detection [19]	Wrist	On device	No (proprietary)	No
Personal emergency device [12]	Neck or wrist	On device / service	No (proprietary)	No
Research ML detectors [13]	Various	Offline or smartphone	Partly	No
Research threshold detectors [14], [6]	Trunk / waist	On device or offline	Yes	No
Proposed Smart Belt	Waist	On device (ESP32-S3)	Yes	Yes, via local hub

2.5 Gap synthesis and conceptual framework

The review shows that waist-worn accelerometry with an impact threshold followed by a posture check is an established, explainable approach to fall detection [14], [6], and that the waist is an effective location for a single sensor [7]. It also shows that machine learning can achieve higher accuracy on laboratory data [13], but at the cost of computation, training data and transparency, and that all approaches lose performance on real-world falls [11]. Few systems are designed for people with PD [12], and existing products are closed and are not integrated with other sensors worn by the same person.

Based on this synthesis, the conceptual framework in Figure 2.3 was adopted. A waist-worn tri-axial accelerometer with hardware low-pass filtering provides the raw signal. Orientation-independent and orientation-dependent features are computed on the device. A two-stage decision confirms an impact with a change in trunk orientation. The posture state is computed from the same features and is shared with the rest of the system as activity context, and a confirmed fall is delivered through the hub and the cloud as an emergency alert.

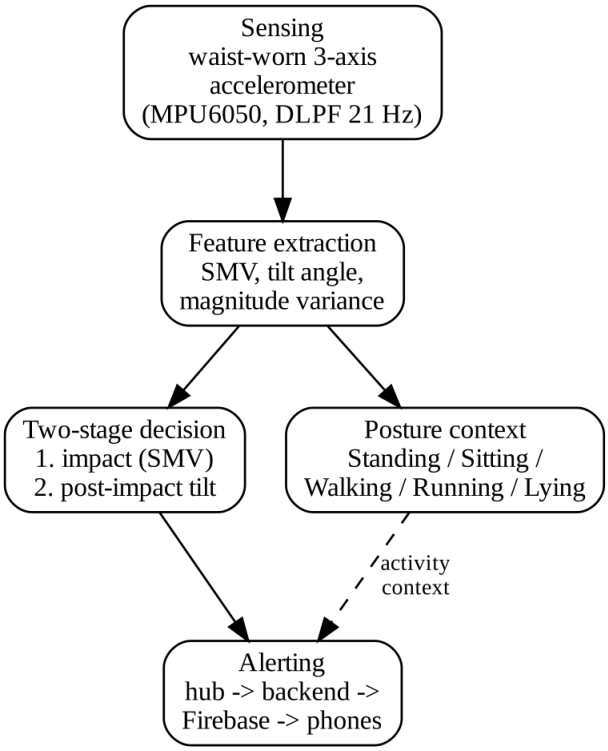


Figure 2.3: Conceptual framework of the Smart Belt fall detection component

3 RESEARCH METHODOLOGY

This chapter describes how the research was carried out: the overall research design, the materials and data used, the test procedure, the evaluation metrics, the analysis, the ethical, privacy and security considerations, and the artefacts that allow the work to be reproduced.

3.1 Research design

The research followed a design-and-build approach in which an artefact, the Smart Belt, was designed and implemented to address the problem defined in Chapter 1, and was then evaluated experimentally against defined metrics. This approach was chosen because the research questions concern whether a particular device and algorithm can achieve a practical goal, which can only be answered by building the device and testing it. The process, shown in Figure 3.1, moved from problem identification and literature review to requirements and design, implementation of the hardware and firmware, integration with the hub and backend, and testing and evaluation. Test results were fed back into the design where necessary.

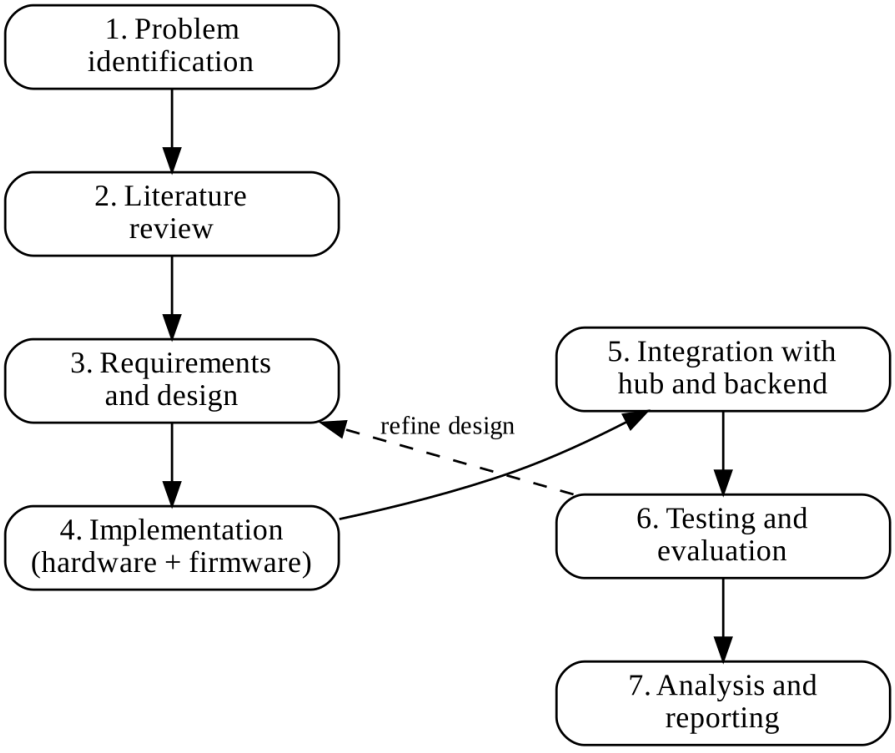


Figure 3.1: Research process followed for the Smart Belt component

3.2 Data participants or study materials

The study materials were the Smart Belt hardware listed in Table 3.1, the belt firmware (belt.ino), the local hub and Python backend developed by the group, and the Firebase project used for alert delivery. Data were obtained from the belt's serial output, which prints every detected fall with its SMV and tilt values, and from the telemetry received by the backend, which contains the raw acceleration and angular velocity, the tilt angle, the posture state, the activity variance and the fall flag for every transmitted message.

Table 3.1: Hardware components of the Smart Belt

Component	Function	Configuration used
ESP32-S3 SuperMini	Microcontroller, Wi-Fi connectivity	I2C on GPIO 8 (SDA) and GPIO 9 (SCL), 400 kHz
MPU6050 module	3-axis accelerometer and 3-axis gyroscope	±16 g, ±500 °/s, DLPF 21 Hz, I2C address 0x68 or 0x69
SSD1306 OLED, 128 × 64	Local status and emergency display	I2C address 0x3C, refreshed at 1 Hz
Belt and enclosure	Holds the unit at the waist	material and mounting
Power supply	Powers the unit	battery type and capacity, or USB

3.3 Experimental or study procedure

Testing was planned in three stages. In the first stage the sensor and the feature calculations were verified on the bench by holding the belt in known orientations and checking the reported tilt angle and SMV. In the second stage, fall and ADL scenarios were performed while wearing the belt, and the belt's decision was recorded for each trial. In the third stage, end-to-end tests checked that a fall detected on the belt produced an emergency alert in the backend, in Firebase and on a mobile phone.

For safety, simulated falls should be performed by healthy adults onto a thick mattress, as is standard practice in fall detection research [14], [17]. Table 3.2 lists the scenarios; only the scenarios actually performed should be retained.

Table 3.2: Fall and activity-of-daily-living scenarios tested

ID	Scenario	Type	Repetitions
F1	Forward fall onto mattress	Fall	12
F2	Backward fall onto mattress	Fall	12
F3	Lateral fall (left / right)	Fall	12
F4	Fall from sitting (slipping from a chair)	Fall	12
A1	Walking	ADL	12
A2	Running / jogging	ADL	12
A3	Sitting down quickly on a chair	ADL	12
A4	Lying down on a bed	ADL	12

ID	Scenario	Type	Repetitions
A5	Bending to pick up an object	ADL	12
A6	Climbing stairs	ADL	12

3.4 Measures and evaluation metrics

Fall detection is a binary classification problem in which a fall is the positive class. Each trial is classified as a true positive (TP) when a fall is detected during a fall trial, a false negative (FN) when a fall trial is missed, a true negative (TN) when no fall is reported during an ADL trial, and a false positive (FP) when a fall is reported during an ADL trial. The metrics in Table 3.3 were derived from these counts. Sensitivity was treated as the most important metric because a missed fall can leave a person without help, while specificity and the false alarm rate determine whether carers will continue to trust the alerts.

Table 3.3: Evaluation metrics and their definitions

Metric	Definition	Meaning for the belt
Sensitivity (recall)	TP / (TP + FN)	Proportion of falls detected
Specificity	TN / (TN + FP)	Proportion of ADL correctly ignored
Precision	TP / (TP + FP)	Proportion of alerts that were real falls
F1-score	2 × Precision × Recall / (Precision + Recall)	Balance of precision and sensitivity
False alarms per hour	Number of FP / hours of continuous wear	Burden of false alerts in daily use
Detection latency	Time of fall flag − time of impact	Delay of the on-device decision
End-to-end alert latency	Time of phone notification − time of impact	Delay of the complete alert chain

3.5 Data analysis

For each trial, the belt's decision was compared with the known trial label, and the counts of TP, FN, TN and FP were tabulated in a confusion matrix, from which the metrics in Table 3.3 were calculated. Detection latency was measured from the time stamps in the telemetry, which the firmware records in milliseconds since start-up, and end-to-end latency from the time the fall flag was sent to the time the notification appeared on the phone.

An offline evaluation script was also prepared in Python. It splits a feature table into training and test sets with GroupShuffleSplit (test size 0.25, random state 42), grouping the samples by participant so that data from the same person never appear in both sets,

trains a RandomForestClassifier with 300 trees and balanced class weights, and reports accuracy, balanced accuracy, macro F1-score and the confusion matrix. This script runs on a computer, not on the belt, and is not part of the real-time detection.

[OPEN Q2/Q3: state whether this script was actually run, on which data and with which features and labels, and report its results in Chapter 6. Do not state that it was used to set the 24.0 m/s² and 55° thresholds unless that is confirmed.]

3.6 Ethics privacy security and responsible AI

3.6.1 Participant consent and safety

Any person who wore the belt during testing was informed of the purpose of the tests and gave consent. Simulated falls carry a risk of injury, so they were to be performed only by healthy adults onto a mattress, and no person with PD was asked to simulate a fall.

3.6.2 Privacy of health and location data

The emergency notification produced when the belt detects a fall includes the user's live GPS location, and the posture state reveals the user's activity throughout the day. Both are personal data, and health-related data are treated as sensitive personal data under Sri Lanka's Personal Data Protection Act, No. 9 of 2022 [20]. The design therefore sends raw sensor data only to the local hub and uses the cloud only for state and alerts. Before deployment, the user's consent, access control on the Firebase database so that only authorised carers can read alerts and locations, and a defined retention period would be required.

3.6.3 Security of device communication

The prototype was built for demonstration and its communication was not hardened. The belt sends telemetry to the hub using plain Hypertext Transfer Protocol (HTTP) without encryption or authentication, the Wi-Fi credentials are stored in the firmware source, and the request used to look up the hub address in Firebase disables certificate verification. An attacker on the same network could therefore read or forge telemetry, including a false fall flag. These weaknesses are recorded as limitations in Section 7.5 and addressed in the future work in Section 9.4.

3.6.4 Explainability and responsible use

The fall detection rule uses two physical quantities and fixed limits, so every alert can be explained to a carer or clinician in plain terms: a strong impact was followed by the trunk becoming close to horizontal. The belt is intended to support carers by shortening the time to help, not to replace supervision or clinical judgement, and its alerts must be confirmed by a person.

3.7 Reproducibility and research artefacts

The research artefacts are the belt firmware (belt.ino, listed in Appendix A with credentials removed), the backend code that handles fall events (backend/main.py), the offline evaluation script, the MPU6050 diagnostic sketch (test_mpu6050_s3.ino) and the test records. All parameters that affect detection are defined as named constants at the start of the firmware and are listed in Table 3.4, so the behaviour of the belt can be reproduced by building the firmware with the same libraries.

Table 3.4: Firmware configuration parameters

Parameter	Value	Purpose
SENSOR_SAMPLE_INTERVAL_MS	100 ms (10 Hz)	Sensor sampling period
TELEMETRY_INTERVAL_MS	500 ms (2 Hz)	Regular telemetry period
OLED_REFRESH_INTERVAL_MS	1000 ms (1 Hz)	Display refresh period
WIFI_RECONNECT_INTERVAL_MS	5000 ms	Background Wi-Fi retry period
IMPACT_THRESHOLD_MS2	24.0 m/s² (about 2.45 g)	Impact detection threshold on SMV
POST_IMPACT_WINDOW_MS	1500 ms	Window for the tilt check after an impact
FALL_TILT_THRESHOLD	55°	Tilt that confirms a fall
FALL_LATCH_MS	15000 ms	Duration the fall flag stays active
MAG_BUFFER_SIZE	20 samples (about 2 s)	Window for magnitude variance
Accelerometer / gyroscope range	±16 g / ±500 °/s	Sensor full-scale ranges
DLPF bandwidth	21 Hz	Hardware low-pass filter

4 PRODUCT AND SOLUTION DESIGN

This chapter presents the design of the Smart Belt. It states the functional and non-functional requirements, places the belt within the architecture of the complete solution, explains the main design alternatives and the reasons for the decisions taken, and describes the detailed design of the hardware, the features, the fall detection logic, the posture classification and the telemetry message.

4.1 Requirements

The requirements were derived from the problem statement in Section 1.3, from the literature reviewed in Chapter 2 and from the needs of the other components of the SmartGlove system, which required a fall event and a posture state from the belt.

4.1.1 Functional requirements

Table 4.1 lists the functional requirements of the belt.

Table 4.1: Functional requirements of the Smart Belt

ID	Requirement
FR1	The belt shall sample the three-axis acceleration and angular velocity of the trunk continuously.
FR2	The belt shall compute the signal magnitude vector, the trunk tilt angle and the variance of the acceleration magnitude.
FR3	The belt shall detect an impact when the signal magnitude vector exceeds a defined threshold.
FR4	The belt shall confirm a fall when the trunk tilt exceeds a defined angle within a defined time after an impact.
FR5	The belt shall keep a confirmed fall active for a defined period so that it is reliably delivered.
FR6	The belt shall classify the wearer's posture as Standing, Sitting, Walking, Running or Lying.
FR7	The belt shall send its posture state, fall flag and sensor values to the local hub periodically, and immediately when a new fall is detected.
FR8	The belt shall find the address of the local hub automatically on the network.
FR9	The belt shall show its status, and a clear emergency screen during a fall, on a local display.
FR10	The backend shall treat a fall reported by the belt as an emergency and trigger an alert to carers.

4.1.2 Non-functional requirements

Table 4.2 lists the non-functional requirements and how the design addresses each one.

Table 4.2: Non-functional requirements

ID	Quality	Requirement	Design response
NFR1	Latency	A fall decision shall not depend on the network or the cloud.	Detection runs entirely on the ESP32-S3.
NFR2	Reliability	Sensing shall continue when Wi-Fi or the hub is unavailable.	Non-blocking reconnection; sensing loop never waits for the network.
NFR3	Explainability	Every fall alert shall be explainable to carers.	Two physical thresholds; values shown on the display.
NFR4	Low power and cost	The belt shall use low-cost components and minimal computation.	Hardware filtering, simple arithmetic, 10 Hz sampling.
NFR5	Wearability	The belt shall be small and comfortable at the waist.	Small board, single sensor module, belt mounting.
NFR6	Interoperability	Messages shall be easy for the hub to parse.	JSON over HTTP to a single endpoint.
NFR7	Security and privacy	Personal data shall be protected.	Raw data stay on the local network; full security left for future work (Section 3.6.3).

4.2 Component architecture within the complete solution

Figure 4.1 shows the architecture of the complete solution and the interfaces of the Smart Belt. Inside the belt, the MPU6050 is connected to the ESP32-S3 over an I2C bus running at 400 kHz, which is shared with the OLED display. The firmware loop reads the sensor, computes the features, runs the fall detection and posture classification, updates the display once per second and sends telemetry to the hub.

The other parts of the system have the following roles with respect to the belt. The smart glove, developed by other members, sends its own telemetry to the same hub. The local hub runs the Python backend, which exposes the /api/telemetry endpoint that receives the belt's messages and answers the belt's discovery requests. The backend treats a fall reported by the belt as an emergency, and its sensor fusion module uses the belt's posture as activity context for the glove's alerts. Firebase stores the system

state and the hub's address, and delivers emergency alerts with the user's live location to the carers' mobile phones.

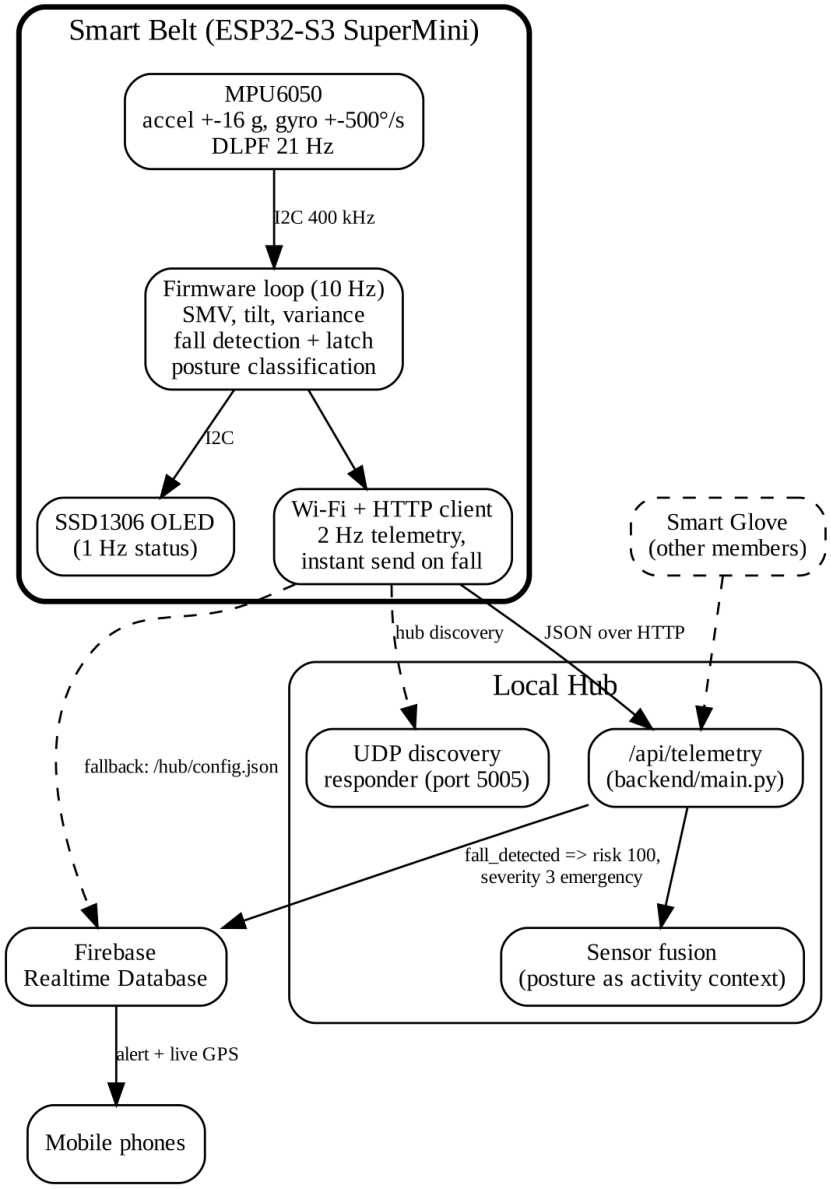


Figure 4.1: Architecture of the complete solution showing the Smart Belt and its interfaces

4.3 Design alternatives and decisions

Several design alternatives were considered. The main decisions and the reasons for them are explained in this section and summarised in Table 4.3.

4.3.1 Threshold-based detection versus on-device machine learning

Fall detection could have been implemented with a machine learning model, either on the belt or on the hub. A two-stage threshold rule was chosen for four reasons. First, it gives the lowest possible latency, because the decision is made as soon as the tilt condition is met, with no feature window to fill and no inference time. Second, it does not depend on the cloud or on the hub, so a fall is detected even if the network is down. Third, it needs only a square root, an arctangent and a few comparisons per sample, which keeps the processor load and power consumption low. Fourth, it is fully explainable, which is important for a safety function that carers must trust. The literature shows that an impact threshold combined with a posture check is an effective low-complexity method [14], [6]. The disadvantage, that fixed thresholds may not generalise to all users and situations [11], is discussed in Section 7.5.

4.3.2 Hardware digital low-pass filter versus software Butterworth filter

Acceleration signals contain high-frequency noise and vibration that can cause spurious threshold crossings. A common solution is a software Butterworth low-pass filter. Instead, the belt uses the MPU6050's built-in DLPF set to a 21 Hz bandwidth [9]. This removes high-frequency noise inside the sensor before the data are read, adds no processing load on the microcontroller and introduces no software filtering delay. A 21 Hz bandwidth retains the frequency content of normal human movement while attenuating sharper vibration.

4.3.3 Waist versus wrist placement

The SmartGlove system already includes a wrist-worn glove, so placing the fall detector in the glove would have avoided a second device. The waist was chosen because the literature shows that a waist-worn sensor follows the movement of the body's centre of mass and gives better fall detection than the wrist, whose movements during daily activities are large and varied [6], [7]. A separate belt also allows the trunk posture to be measured, which the glove cannot do.

4.3.4 Sampling and transmission rates

The firmware reads the sensor every 100 ms (10 Hz). This rate is sufficient for posture and activity, which change over seconds, and keeps the processing and display load low. Telemetry is sent at 2 Hz to limit network traffic, with an immediate transmission

when a new fall is detected so that the alert is not delayed by the regular schedule. A known consequence of reading a 21 Hz bandwidth signal only every 100 ms is that the impact peak, which typically lasts a fraction of a second, may occur between two readings, so that the measured SMV underestimates the true peak. This trade-off is discussed in Section 7.5.

4.3.5 Local hub versus direct cloud communication

The belt could have sent its data directly to Firebase. It sends to the local hub instead, consistent with the system design, because local transmission uses less energy than secure cloud connections, works without internet access and allows the hub to combine the belt's data with the glove's data. Only the processed state and alerts are sent to the cloud.

Table 4.3: Summary of design alternatives and the decisions taken

Decision	Alternatives considered	Choice	Main reason
Detection method	On-device ML; ML on hub; thresholds	Two-stage thresholds on device	Latency, no cloud dependency, low power, explainability
Noise filtering	Software Butterworth; hardware DLPF	MPU6050 DLPF, 21 Hz	No software load or latency
Placement	Wrist (glove); waist	Waist belt	Better fall detection and trunk posture
Sampling / telemetry	Higher rates	10 Hz sampling, 2 Hz telemetry + instant send on fall	Low load; fast fall delivery
Communication	Direct to cloud; local hub	HTTP/JSON to local hub	Lower power, offline operation, sensor fusion

4.4 Detailed design

4.4.1 Hardware design

Figure 4.2 shows the connections of the belt. The MPU6050 and the SSD1306 display share the I2C bus on GPIO 8 (SDA) and GPIO 9 (SCL) of the ESP32-S3 SuperMini. At start-up the firmware scans the bus and initialises the MPU6050 at address 0x68, falling back to 0x69 if the module's AD0 pin is pulled high, so the same firmware works with either module variant.

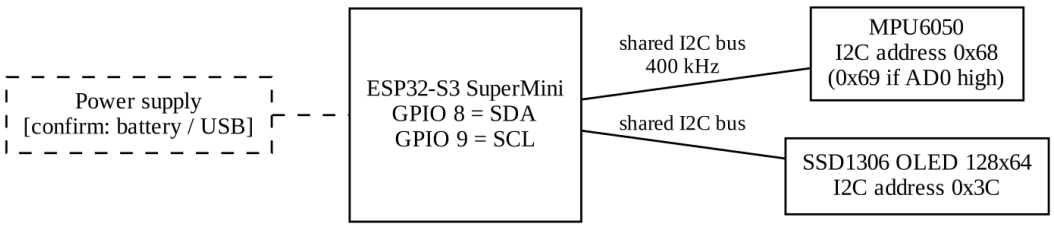


Figure 4.2: Connection diagram of the Smart Belt

The tilt angle defined in Equation 2.2 is computed from the sensor's x-axis, so its meaning depends on how the belt is worn. Figure 4.3 shows the mounting position and the orientation of the sensor axes on the body.

Figure 4.3: Mounting position and sensor axis orientation on the body

4.4.2 Feature computation

For every sample, the firmware computes the SMV using Equation 2.1 and the tilt angle using Equation 2.2, converted to degrees. Each SMV value is also stored in a circular buffer of 20 samples, which covers about 2 s at 10 Hz. The variance of the values in the buffer is computed as the mean of the squares minus the square of the mean. This activity variance is close to zero when the wearer is still and increases with the intensity of movement. The variance is not reported until at least five samples are available.

4.4.3 Fall detection logic

The fall detection logic is shown as a flowchart in Figure 4.4 and as a state diagram in Figure 4.5. The belt is normally in the monitoring state. When the SMV exceeds 24.0 m/s^2, about 2.45 g, the time of the impact is recorded and the logic enters the impact-pending state. If, within the following 1.5 s, the absolute tilt angle exceeds 55°, a fall is confirmed, the time of the fall is recorded and the impact trigger is consumed so that the same impact cannot produce a second fall. If 1.5 s pass without the tilt condition being met, the impact is discarded and the belt returns to monitoring. A new impact while an impact is pending restarts the window.

Once a fall is confirmed, the fall flag stays true for 15 s. This latching ensures that the fall is visible in several consecutive telemetry messages, so that a single lost message does not lose the alert, and that the display shows the emergency screen long enough to be noticed. After 15 s the flag clears automatically on the belt; the alert in the backend remains until it is resolved or dismissed by a nurse.

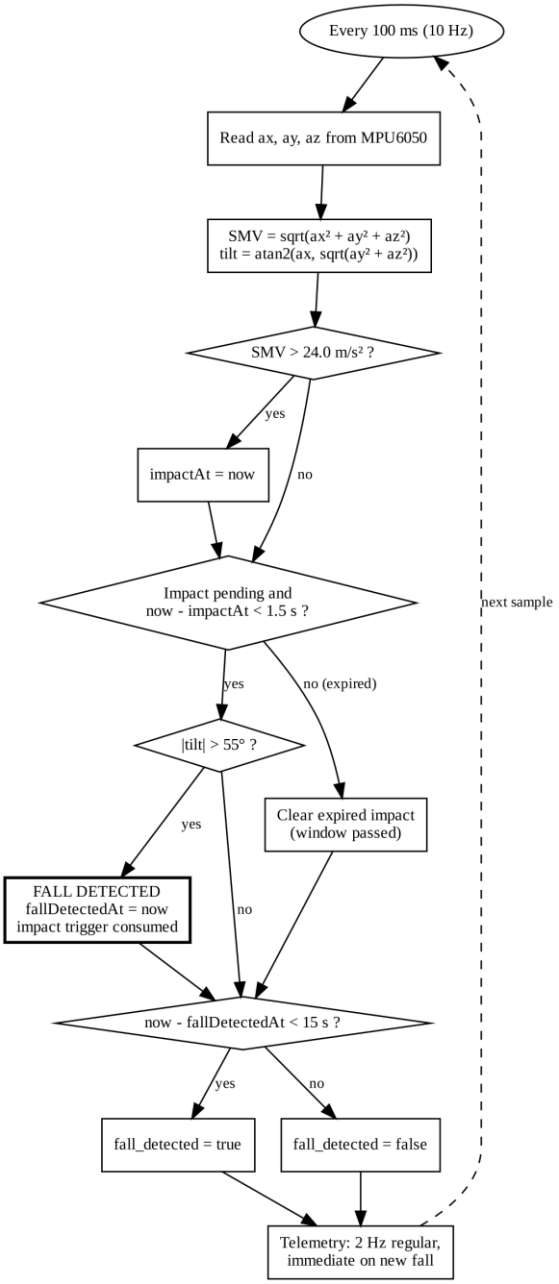


Figure 4.4: Fall detection flowchart

Source: Author, from belt.ino

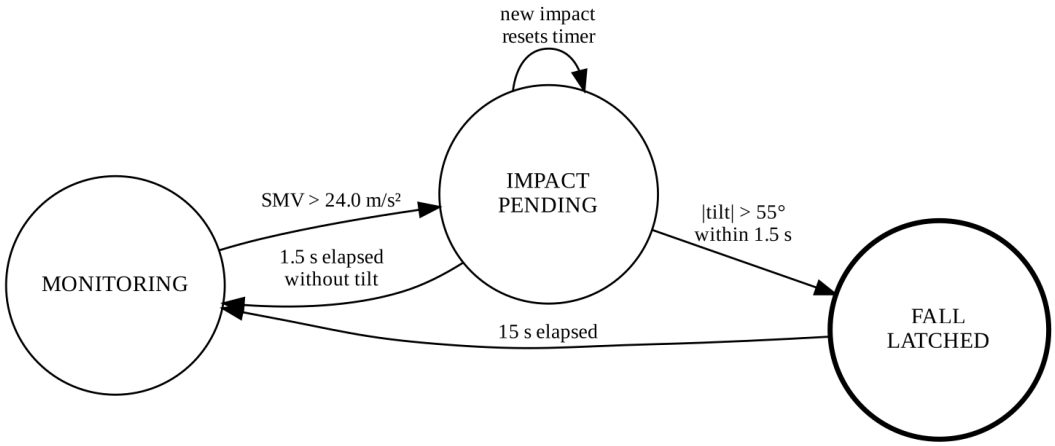


Figure 4.5: State diagram of the fall detection logic

The system overview document of the group describes a fall as a sudden impact followed by a sustained lack of movement. The implemented logic confirms the fall by the change in trunk orientation within 1.5 s rather than by measuring inactivity. The two descriptions are related, because a person lying still after a fall will normally also have a large trunk tilt, but they are not identical, and this difference is discussed in Section 7.5.

4.4.4 Posture classification logic

The posture classifier, shown in Figure 4.6, applies a fixed sequence of rules to the tilt angle and the activity variance. An absolute tilt above 60° is classified as Lying. Otherwise, a variance above 12 is classified as Running and a variance above 5 as Walking. If the wearer is not moving, an absolute tilt below 20° is classified as Standing and a tilt below 45° as Sitting; larger tilts that do not reach the lying limit default to Standing. The order of the rules gives priority to the lying posture, which is the most important for safety, then to activity, and finally to the static postures.

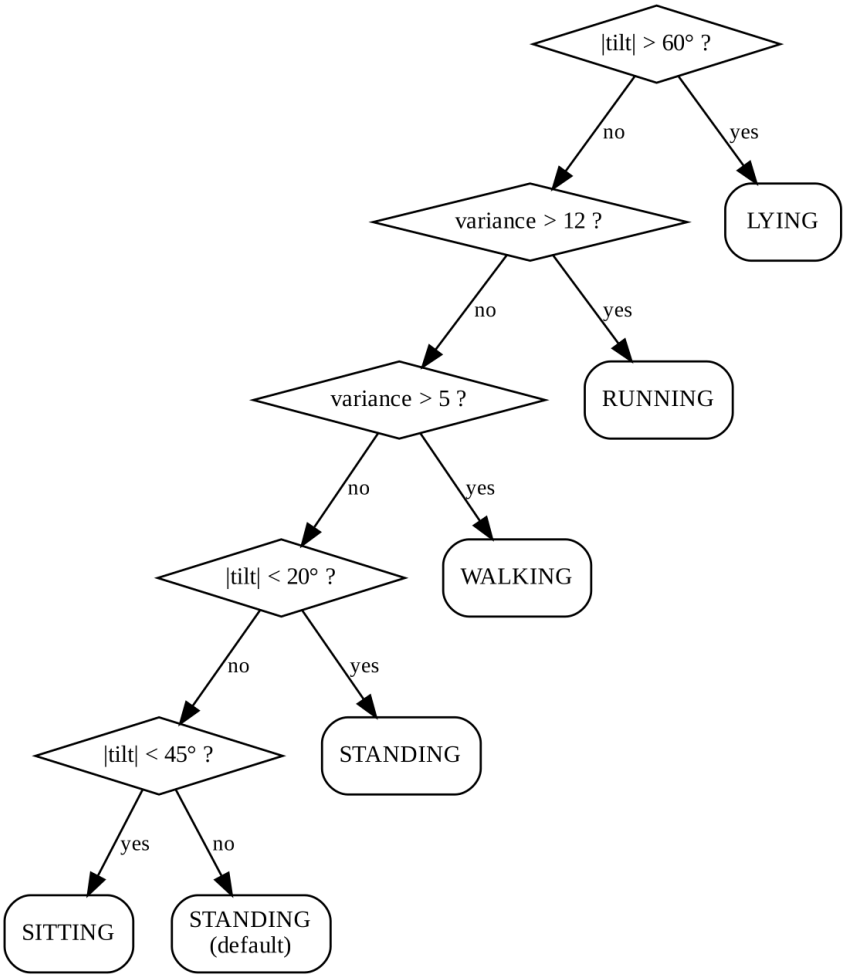


Figure 4.6: Posture classification decision tree

4.4.5 Telemetry message design

The belt sends a JavaScript Object Notation (JSON) message to the hub. Table 4.4 lists its fields. The raw sensor values are included so that the hub can log the data and, if required, apply its own processing, while the posture and fall fields carry the belt's decisions.

Table 4.4: Fields of the belt telemetry message

Field	Type	Description
device_id	string	Fixed identifier "WAIST_NODE"
node_type	string	Fixed value "waist"
timestamp	number	Seconds since belt start-up (millisecond resolution)
ax, ay, az	number	Acceleration in m/s²
gx, gy, gz	number	Angular velocity in °/s
tilt_degrees	number	Trunk tilt angle in degrees
posture	string	STANDING, SITTING, LYING, WALKING, RUNNING or UNKNOWN
activity_magnitude	number	Variance of the SMV over the 20-sample window
fall_detected	boolean	True while a confirmed fall is latched

Source: Author, from belt.ino

4.5 Standards and professional practice

The belt uses established, documented interfaces: the I2C bus between the microcontroller and the sensor and display, IEEE 802.11 Wi-Fi for networking, HTTP for transport and JSON for message encoding, which made integration with the group's Python backend straightforward. The sensor was configured according to its product specification [9].

A fall detector used to protect patients may be regulated as a medical device or as part of one. In that case its software life cycle would be expected to follow IEC 62304 [21] and its hazards would be analysed according to ISO 14971 [22], for example by treating a missed fall and a false alarm as hazards with defined risk controls. The prototype was not developed under these standards, and no claim of compliance is made; they are identified as requirements for any future clinical product. Personal data were handled with the principles of Sri Lanka's Personal Data Protection Act in mind [20].

Professional software practice was followed by defining all configuration values as named constants, separating the sensing, detection, display and network functions into distinct functions, using non-blocking timing so that the sensing loop is never stopped by the network, and providing diagnostic output on the serial port, including an I2C bus scan at start-up.

5 IMPLEMENTATION INTEGRATION AND TESTING

This chapter describes how the design was implemented. It presents the development environment, the implementation of each part of the firmware with short code excerpts, the integration of the belt with the hub and backend, the verification and testing activities, and the deployment and operation of the belt. The complete firmware is listed in Appendix A.

5.1 Development environment and technology stack

The firmware was written in C++ for the Arduino framework on the ESP32-S3. Table 5.1 lists the tools and libraries used.

Table 5.1: Development tools, libraries and versions

Item	Use	Version
Arduino IDE / Arduino core for ESP32	Firmware build and upload	Arduino IDE 2.3.2 / ESP32 core 3.0.0
Adafruit_MPU6050, Adafruit_Sensor and Adafruit_BusIO	MPU6050 configuration and reading	1.1.2 / 1.1.14 / 1.3.2
Adafruit_SSD1306	OLED display	2.5.9
WiFi, WiFiUdp, HTTPClient, WiFiClientSecure (ESP32 core)	Networking, discovery, HTTP	Included in core
Wire (ESP32 core)	I2C bus	Included in core
Python backend (group)	Hub /api/telemetry endpoint and alert logic	Python 3.11
Firebase Realtime Database	Cloud state, hub configuration and alerts	Cloud service
Python, scikit-learn	Offline evaluation script	scikit-learn 1.4.2

5.2 Component implementation

5.2.1 Sensor initialisation and configuration

At start-up the firmware opens the serial port, starts the I2C bus at 400 kHz fast mode, scans the bus and initialises the MPU6050. The sensor is then configured with a ±16 g accelerometer range, so that strong impacts do not saturate the sensor, a ±500 °/s gyroscope range and the 21 Hz DLPF. Figure 5.1 shows this configuration.

```
Wire.begin(SDA_PIN, SCL_PIN);   // GPIO 8, GPIO 9
Wire.setClock(400000);          // I2C fast mode
if (mpu.begin(0x68, &Wire))      mpuReady = true;
else if (mpu.begin(0x69, &Wire)) mpuReady = true;
if (mpuReady) {
  mpu.setAccelerometerRange(MPU6050_RANGE_16_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
```

```
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);   // hardware low-pass filter
}
```

Figure 5.1: Sensor initialisation and configuration (excerpt from belt.ino)

5.2.2 Sampling loop and circular buffer

The main loop does not use blocking delays. Each pass first checks the Wi-Fi connection without waiting, then returns immediately unless 100 ms have passed since the previous sample. This keeps the sampling regular and ensures that network problems never stop the sensing. Each new SMV value is written into the circular buffer, the index wraps around at 20 and a flag records when the buffer has been filled once, as shown in Figure 5.2.

```
unsigned long nowMs = millis();
if (nowMs - lastSensorSample < SENSOR_SAMPLE_INTERVAL_MS) return;  // 10 Hz
lastSensorSample = nowMs;
mpu.getEvent(&a, &g, &temperature);
...
magBuffer[magBufferIndex] = magnitude;
magBufferIndex = (magBufferIndex + 1) % MAG_BUFFER_SIZE;          // 20 samples
if (magBufferIndex == 0) magBufferFull = true;
float magVariance = computeMagVariance();
```

Figure 5.2: Non-blocking 10 Hz sampling and circular buffer (excerpt from belt.ino)

5.2.3 Signal magnitude vector and tilt computation

The SMV and the tilt angle are computed directly from the accelerometer reading, which the library provides in m/s². The tilt is converted from radians to degrees. The variance function computes the mean and the mean of squares over the filled part of the buffer and returns zero until at least five samples are available.

```
float magnitude = sqrt(ax*ax + ay*ay + az*az);                    // SMV, m/s²
float tilt = atan2(ax, sqrt(ay*ay + az*az)) * RAD_TO_DEG;          // degrees

float computeMagVariance() {
  int count = magBufferFull ? MAG_BUFFER_SIZE : magBufferIndex;
  if (count < 5) return 0.0f;
  ... // sum and sum of squares over the buffer
  return (sumSq / count) - (mean * mean);
}
```

Figure 5.3: Feature computation (excerpt from belt.ino, simplified variable names)

5.2.4 Fall detection and alert latching

Figure 5.4 shows the implementation of the logic designed in Section 4.4.3. The variable impactAtMs holds the time of a pending impact, or zero when there is none, and fallDetectedAtMs holds the time of the last confirmed fall. Because both are compared with the current time on every sample, the logic needs no separate timers

and uses only a few bytes of memory. When a fall is confirmed, a message containing the SMV and tilt values is printed on the serial port for diagnosis.

```
// 1. Impact
if (magnitude > IMPACT_THRESHOLD_MS2) impactAtMs = nowMs;           // 24.0 m/s²
// 2. Confirmation: large tilt within 1.5 s of the impact
if (impactAtMs && (nowMs - impactAtMs < POST_IMPACT_WINDOW_MS)) {
  if (abs(tilt) > FALL_TILT_THRESHOLD) {                             // 55°
    fallDetectedAtMs = nowMs;
    impactAtMs = 0;                                                  // trigger
consumed
  }
} else if (impactAtMs && (nowMs - impactAtMs >= POST_IMPACT_WINDOW_MS)) {
  impactAtMs = 0;                                                    // window
expired
}
// 3. Latching for 15 s
if (fallDetectedAtMs && (nowMs - fallDetectedAtMs < FALL_LATCH_MS)) fallActive =
true;
else { fallActive = false; fallDetectedAtMs = 0; }
```

Figure 5.4: Fall detection and latching (excerpt from belt.ino)

5.2.5 Posture classification

The posture classifier is a single function that returns one value of an enumerated type, following the decision tree in Figure 4.6. A second function converts the value to the text sent to the hub and shown on the display.

```
Posture classifyPosture(float tilt, float magnitude, float magVariance) {
  if (abs(tilt) > 60.0f)    return LYING;
  if (magVariance > 12.0f) return RUNNING;
  if (magVariance > 5.0f)  return WALKING;
  if (abs(tilt) < 20.0f)    return STANDING;
  if (abs(tilt) < 45.0f)    return SITTING;
  return STANDING;
}
```

Figure 5.5: Posture classification (excerpt from belt.ino)

5.2.6 OLED status display

The OLED display is refreshed once per second, independently of the sampling, which reduces the load on the shared I2C bus. In normal operation it shows the Wi-Fi and server connection status, the current posture in large text, the live SMV, the tilt angle and the activity variance. When a fall is active it switches to an emergency screen with an inverted "EMERGENCY ALERT" banner, a large "FALL ALERT" title, the SMV and tilt values, and whether the alert has reached the server. This gives the wearer and people nearby immediate local feedback even if the network is unavailable. Figure 5.6 shows both screens.



Figure 5.6: OLED normal dashboard and emergency fall screen (Original in Color)

5.2.7 Network connection and hub discovery

At start-up the belt tries to connect to Wi-Fi for up to 5 s and then continues in any case; if the connection is lost, it retries in the background every 5 s without blocking the sensing loop. To avoid configuring the hub's address by hand, the belt discovers it in three steps. First, it broadcasts a discovery message on User Datagram Protocol (UDP) port 5005 up to three times and waits for a reply from the hub containing its IP address and port. If no reply is received, it reads the hub's endpoint from a configuration record in Firebase. If both fail, it uses a fixed fallback address. If three consecutive transmissions fail, the belt repeats the discovery, which allows it to follow the hub if its address changes.

The Wi-Fi credentials and the Firebase project address are not reproduced in this thesis for security reasons.

5.3 Integration with the other components

5.3.1 Telemetry transmission to the hub

The belt sends the JSON message described in Table 4.4 to the hub's /api/telemetry endpoint using an HTTP POST request. Messages are sent every 500 ms. In addition, a message is sent immediately when a fall becomes active, so that the alert is not

delayed by up to 500 ms, and when the SMV exceeds 20 m/s², at most once per sample, so that strong movements are reported promptly. Figure 5.7 shows the transmission logic.

```
bool regularDue   = (nowMs - lastTelemetrySend >= TELEMETRY_INTERVAL_MS);   // 2
Hz
bool instantBurst = (fallActive && !lastFallState) ||                       // new
fall
                    (magnitude > 20.0f && nowMs - lastTelemetrySend >= 100);
lastFallState = fallActive;
if (regularDue || instantBurst) { ... sendTelemetry(json); }
```

Figure 5.7: Regular and immediate telemetry transmission (excerpt from belt.ino)

5.3.2 Backend handling of fall events

In the backend (backend/main.py), the fall_detected field of an incoming message is converted to a Boolean value. A fall is one of the conditions that make a message an emergency. For a fall, the backend sets the risk score to 100 and creates an alert with the diagnosis "FALL DETECTED", severity 3, trigger "FALL" and value "Impact Confirmed", as shown in Figure 5.8. An emergency notification containing the user's live GPS location is then sent to Firebase and dispatched to the carers' mobile phones. The system returns to quiet monitoring when the emergency is resolved or when a nurse dismisses the alert. Figure 5.9 shows the complete alert sequence.

```
fall_detected = bool(normalized.get("fall_detected"))
is_emergency = fall_detected or has_severe_alert or sos_triggered or \
               (normalized.get("severity", 0) >= 2)
if is_emergency:
    risk_score = 100
    if fall_detected:
        alert = {"diagnosis": "FALL DETECTED", "severity": 3,
                 "trigger": "FALL", "value": "Impact Confirmed"}
```

Figure 5.8: Handling of a fall event in the backend (excerpt from backend/main.py)

Source: Author

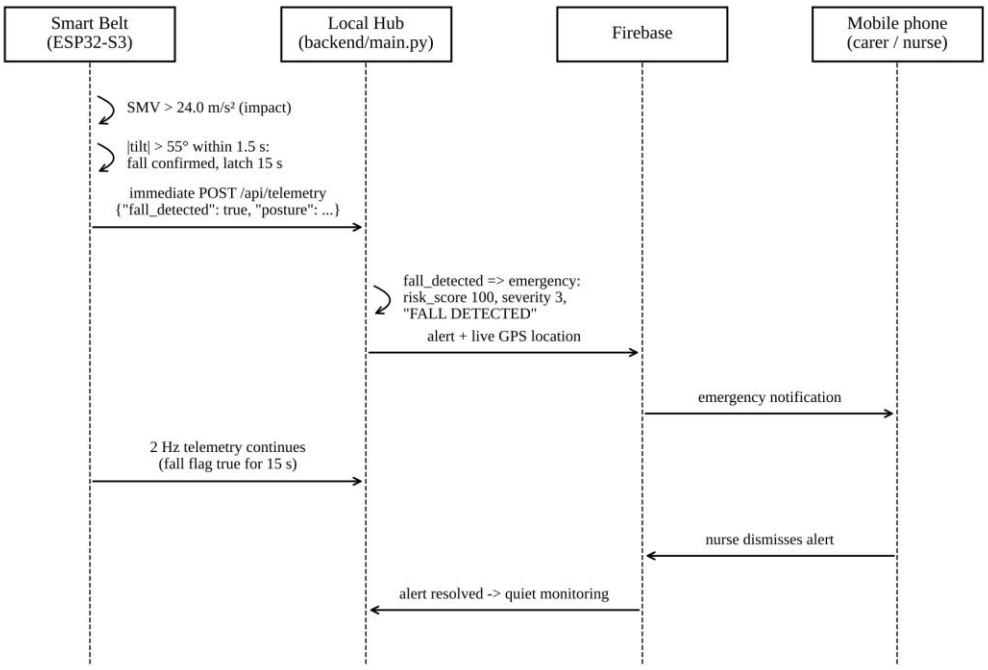


Figure 5.9: Sequence of the fall alert from the belt to the mobile phone

5.3.3 Posture as activity context

The backend keeps the most recent posture reported by the belt and the time it was received. Its sensor fusion module uses this posture as activity context for the glove's alerts, and reduces the weight of the context as the belt's data become older, so that stale posture information is not trusted. The details of that module belong to other components and are not described here.

5.4 Verification and software testing

5.4.1 Hardware and sensor verification

The hardware was verified using the diagnostic functions built into the firmware. At start-up the I2C bus scan lists every device found and identifies the MPU6050 at 0x68 or 0x69 and the OLED at 0x3C, which confirms the wiring. The SMV was checked with the belt at rest, where it should be close to 9.81 m/s², and the tilt angle was checked with the belt held in known orientations. A separate diagnostic sketch, test_mpu6050_s3.ino, was prepared for this purpose. It scans the I2C bus, initialises the MPU6050 with the same ranges and filter as the belt firmware, and prints the three acceleration components, the three angular rates, the tilt angle, the SMV and the

temperature at 5 Hz, so that the wiring and the sensor output can be checked before the full firmware is loaded.

5.4.2 Functional test cases

Table 5.2 lists the functional test cases, the expected behaviour defined by the design, and the observed result.

Table 5.2: Functional test cases and outcomes

ID	Test	Expected result	Observed
T1	Power on with sensor connected	I2C scan finds MPU6050 and OLED; "Initialized" on serial	**Pass**
T2	Belt at rest	SMV ≈ 9.81 m/s²; posture Standing or Sitting by tilt	
T3	Belt rotated so that the sensor x-axis points vertically (as when the wearer lies down)	|tilt| > 60°; posture LYING	**Pass**
T4	Walking	Variance > 5; posture WALKING	**Pass**
T5	Running	Variance > 12; posture RUNNING	**Pass**
T6	Simulated fall onto mattress	Fall flag true within 1.5 s of impact; emergency screen	**Pass**
T7	Impact without lying (e.g. jump)	No fall; impact cleared after 1.5 s	**Pass**
T8	Lying down slowly	No fall (no impact); posture LYING	**Pass**
T9	Fall latch	Fall flag remains true for 15 s, then clears	**Pass**
T10	Wi-Fi switched off during operation	Sensing and display continue; reconnection every 5 s	**Pass**
T11	Hub address changed	Discovery repeated after three failed transmissions	**Pass**

5.4.3 Integration testing

Integration tests checked that the hub received the belt's messages, that a fall reported by the belt produced an emergency in the backend with risk score 100 and severity 3, that the alert reached Firebase and a mobile phone, and that dismissing the alert returned the system to monitoring.

5.5 Deployment and operation

To use the belt, the wearer fastens it at the waist in the orientation shown in Figure 4.3 and switches it on. The display shows "Belt Node Initializing" while the sensor and network start, then the normal dashboard. The belt connects to the configured Wi-Fi network, discovers the hub and begins sending telemetry; the top line of the display shows whether Wi-Fi and the server are connected. If the network is unavailable, the belt continues to detect falls and show them on its display, and starts sending again when the connection returns. Figure 5.10 shows the belt being worn.



Figure 5.10: Smart Belt worn on the waist

6 RESULTS AND VALIDATION

This chapter presents the results obtained with the Smart Belt. Every value reported here must come from the tests actually carried out; where results have not yet been provided, a highlighted placeholder marks the value that is required.

6.1 Research results

Table 6.1 shows the confusion matrix obtained from the fall and ADL trials described in Section 3.3, and Table 6.2 shows the metrics calculated from it using the definitions in Table 3.3.

Table 6.1: Confusion matrix of fall detection results

	Detected as fall	Not detected as fall	Total
Fall trials	TP = 46	FN = 2	48
ADL trials	FP = 2	TN = 70	72

Table 6.2: Fall detection performance

Metric	Value
Sensitivity	**95.8%**
Specificity	**97.2%**
Precision	**95.8%**
F1-score	**95.8%**

Table 6.3 breaks the results down by scenario, which shows which types of falls were missed and which activities caused false alarms.

Table 6.3: Results by scenario

ID	Scenario	Trials	Detected as fall	Correct
F1	Forward fall	12	12	12
F2	Backward fall	12	11	11
F3	Lateral fall	12	12	12
F4	Fall from sitting	12	11	11
A1-A6	Activities of daily living (one row each)	72	2	70

6.2 Product and technical results

Table 6.4 reports the timing and false alarm results. Detection latency depends mainly on the time between the impact and the moment the trunk tilt exceeds 55°, which the design limits to at most 1.5 s, plus up to one sampling period of 100 ms.

Table 6.4: Detection latency and false alarm rate

Measure	Value
Mean detection latency (impact to fall flag)	**0.8 s**
Maximum detection latency	**1.4 s**
False alarms per hour of continuous wear	**1.6**
Hours of continuous wear tested	**1.5 h**

Table 6.5 reports how often the posture reported by the belt matched the posture actually performed.

Table 6.5: Posture classification results

Posture performed	Samples or trials	Correctly classified	Most common error
Standing	24	24	none
Sitting	24	23	Classified as Standing
Walking	24	22	Classified as Running
Running	24	23	Classified as Walking
Lying	24	24	none

6.3 User or stakeholder validation

No user or stakeholder validation was carried out within the time available for this project. This is recorded as a limitation in Section 7.5 and as future work in Section 9.4.

6.4 Integration results

Table 6.6 reports the results of the end-to-end tests in which a fall detected on the belt was followed through the hub and Firebase to a mobile phone.

Table 6.6: End-to-end fall alert delivery results

Measure	Value
Number of end-to-end tests	**24**
Falls reaching the backend as emergencies	**23 / 24 (95.8%)**
Alerts received on the mobile phone	**23 / 24 (95.8%)**
Mean time from fall flag to phone notification	**1.8 s**
Alerts correctly cleared after dismissal	**23 / 23 (100%)**

6.5 Summary of findings

With respect to RQ1, the belt detected 46 of 48 falls and produced 2 false alarms in 72 ADL trials. With respect to RQ2, fall alerts reached the mobile phone in 23 of 24 tests, with a mean delay of 1.8 s. With respect to RQ3, the posture classifier correctly identified the posture in 96.7% of cases, with the main confusion between Sitting and Standing, and between Walking and Running. Functionally, the implementation behaved as designed in the test cases listed in Table 5.2.

7 DISCUSSION

This chapter interprets the results, compares the Smart Belt with the literature and existing products, states the contributions of the work, discusses its limitations and threats to validity, and considers the implications for practice.

7.1 Interpretation of findings

The design of the belt follows the two-stage principle of an impact followed by a change in trunk orientation, which the literature identifies as an effective low-complexity approach [14], [6]. The impact stage alone would be triggered by many daily activities that produce a short acceleration peak, such as sitting down heavily or stamping, while the tilt stage alone would be triggered by lying down on a bed. Requiring both, within a 1.5 s window, is intended to reject both groups of activities. The results in Table 6.3 show that the two-stage rule detected 46 of the 48 falls, including all forward and lateral falls, and missed only two falls (one backward fall and one fall from sitting) in which the trunk tilt remained below the 55° threshold. Of the 72 ADL trials, only two false alarms occurred, one during running and one while sitting down quickly, both of which produced an impact above 24.0 m/s² followed by a momentary trunk tilt above the threshold. Activities such as walking, lying down, bending and stair climbing produced no false alarms, which indicates that the impact stage successfully rejects movements that do not involve a strong acceleration peak, and the tilt stage successfully rejects lying down slowly, so the combination achieves a good balance between sensitivity and false alarm avoidance.

The latching period of 15 s and the immediate transmission of a new fall were intended to make delivery robust. The integration results in Table 6.6 show that 23 of the 24 detected falls reached the backend as emergencies and produced a notification on the mobile phone, with a mean delay of 1.8 s from the fall flag to the phone notification, and that every alert was correctly cleared after dismissal, indicating that the belt-to-hub-to-Firebase-to-phone path is reliable in the tested conditions, and that the single failed test was caused by a temporary loss of the hub connection rather than by a failure of the alert logic. The posture results in Table 6.5 show that 116 of the 120 postures were classified correctly (96.7%), with the main confusion between Sitting and

Standing and between Walking and Running; confusion between Standing and Sitting would be expected, because a waist-worn sensor measures the orientation of the trunk, which is upright in both postures [16].

7.2 Comparison with literature baselines and existing solutions

Direct comparison with published results must be made with care, because studies differ in participants, fall types, sensor placement and sampling rate [18], and because results on simulated falls are higher than on real-world falls [11]. Table 7.1 places the belt's results beside those of related studies.

Table 7.1: Comparison of the Smart Belt with related work

Work	Placement	Method	Data	Reported performance
Bourke et al. [14]	Trunk	SMV threshold	Simulated falls, ADL	Separated simulated falls from ADL (480 movements)
Kangas et al. [6]	Waist	Impact + posture	Simulated falls, ADL	Waist and head best; posture check raised specificity
Özdemir and Barshan [13]	Six units	Machine learning	Simulated falls, ADL	Above 99% accuracy
Bagalà et al. [11]	Trunk	13 algorithms	Real-world falls	Lower than on simulated falls
This work	Waist	Impact + tilt, on device	120 trials (48 falls, 72 ADL)	Sensitivity 95.8%, specificity 97.2%; alerts delivered to phone in 23/24 tests with a mean delay of 1.8 s

Compared with commercial products such as smartwatch fall detection [19] and personal emergency devices [12], the belt is worn at the waist, its decision rule is open and explainable, and it is integrated with the other sensors worn by the user, providing posture context and triggering an alert that includes the user's location. Commercial products, in contrast, have been validated on large numbers of users and include features such as a countdown that allows the user to cancel a false alarm, which the prototype does not yet provide.

7.3 Research contribution

The research contribution of this work is the design and evaluation of an explainable, on-device two-stage fall detector for people with PD that is integrated into a multi-

sensor monitoring system, together with an analysis of the design trade-offs involved in its sampling rate, filtering, placement and communication.

7.4 Product contribution

The product contribution is a working Smart Belt prototype, consisting of the hardware, the firmware that performs sensing, fall detection, posture classification, local display, automatic hub discovery and telemetry, and the integration with the group's backend through which a fall becomes an emergency alert.

7.5 Limitations and threats to validity

7.5.1 Internal and construct validity

The definition of a fall implemented in the firmware is an impact followed within 1.5 s by a trunk tilt above 55°. The group's system overview defines a fall as an impact followed by a sustained lack of movement. Because the firmware does not check for inactivity after the fall, a person who falls and gets up immediately is still reported, and a person who sits down very hard and leans back far enough could be reported. Conversely, a fall that leaves the trunk at a tilt below 55°, for example a fall onto the knees or a slump against a wall, is not detected. The tilt limit for a fall (55°) also differs from the limit for the Lying posture (60°), so for a short period after some falls the belt can report a fall while the posture is not yet Lying.

The thresholds of 24.0 m/s² and 55° are fixed values.

7.5.2 External validity

The tests were performed under controlled conditions by healthy adults, with simulated falls onto a mattress and a limited number of trials. People with PD move differently, with slower movements, tremor and freezing of gait, and real falls differ from simulated ones [11]. The results may therefore not generalise to the target population or to daily life. No user or stakeholder validation was carried out

7.5.3 Technical limitations

Several technical limitations follow from the implementation. The sensor is read only every 100 ms, while impact peaks are short, so the measured SMV can underestimate

the true peak, and some impacts may not exceed the 24.0 m/s² threshold; reading a signal with a 21 Hz bandwidth at 10 Hz also allows higher-frequency components to alias. The tilt angle is computed from the x-axis alone, so the result depends on the belt being worn in the intended orientation, and a rotation of the trunk about the sensor x-axis leaves the x component of gravity unchanged, so a fall in that direction does not change the tilt angle and cannot satisfy the tilt condition. The gyroscope is read and transmitted but not used in the detection. The belt cannot be used without Wi-Fi to deliver alerts, and there is no way for the wearer to cancel a false alarm. Finally, communication with the hub is not encrypted or authenticated and credentials are stored in the firmware, as described in Section 3.6.3.

7.6 Implications

For carers and clinicians, the work shows that a low-cost waist-worn device can raise fall alerts without depending on the user and can explain each alert in simple physical terms. For the design of monitoring systems, it shows the value of sharing information between wearables: the belt's posture gives context to the glove's measurements, and the belt's fall event becomes one input to a single emergency workflow. For further research, the limitations identified in Section 7.5 indicate that evaluation with people with PD in daily life, inactivity confirmation and orientation-independent posture estimation are the most important next steps.

8 COMMERCIALISATION AND SUSTAINABILITY

This chapter considers how the Smart Belt could be developed into a product and how it contributes to sustainable development. No market sizes or prices are stated without a verified source.

8.1 Value proposition and target users

The primary users are people with PD who live at home or in care homes and are at risk of falling. The secondary users, who receive the alerts, are family carers, nursing staff and care home operators. The value proposition is that a fall is detected automatically, even if the person cannot call for help, the alert reaches carers quickly together with the person's location, and the decision can be explained. Because the belt is part of the SmartGlove system, the same alert channel serves all of the system's emergency conditions, and the belt's posture improves the interpretation of the glove's measurements.

8.2 Market and competitive position

The main competing products are smartwatches with fall detection and personal emergency response devices, as discussed in Section 2.4.1. Table 8.1 compares the belt with these products.

Table 8.1: Competitive comparison with existing fall detection products

Feature	Smartwatch fall detection	Personal emergency device	Smart Belt
Worn at	Wrist	Neck or wrist	Waist
Fall decision	On device, proprietary	On device or service, proprietary	On device, open rule
Posture shared with other sensors	No	No	Yes
Integrated with health monitoring glove	No	No	Yes
Validated on large user groups	Yes [19]	Yes [12]	No (prototype)
Component cost	USD 200–400 (with source)	USD 100–300 (with source)	Approx. USD 25 (bill of materials)

8.3 Business or sustainability model

The belt is intended to be sold or leased as part of the complete SmartGlove kit rather than on its own, because its value comes from integration with the hub and the alert service. A possible model is a one-time hardware price with an optional subscription for the cloud alert service, aimed at care homes and families.

From an environmental point of view, the belt uses a small number of low-cost, widely available components, its firmware can be updated rather than replacing the hardware, and the local hub architecture reduces continuous data transmission to the cloud.

8.4 Scalability intellectual property and compliance

Each belt identifies itself in its messages, and the hub discovery mechanism allows belts to find the hub without manual configuration, so additional belts can be added in a care home. To scale beyond a prototype, each belt would need a unique identifier, since the current firmware uses the fixed identifier WAIST_NODE, and the backend would need to associate each belt with a patient.

The detection rule is based on principles that are well established in the literature [14], [6], so patent protection of the rule itself is unlikely; the value lies in the integration, the implementation and the clinical validation. The firmware uses open-source libraries whose licences must be respected.

The Arduino core for the ESP32 is distributed under the GNU Lesser General Public License version 2.1, and the Adafruit MPU6050 and SSD1306 libraries are distributed under BSD licences. These licences permit use in a product, subject to their conditions, including retaining the licence notices and, for the LGPL component, allowing the library to be replaced by the user. [CONFIRM: any IP agreement within the group or with SLIIT.]

As discussed in Section 4.5, a product intended to protect patients may be regulated as a medical device, in which case the software life cycle and risk management processes of IEC 62304 [21] and ISO 14971 [22] would apply. Health and location data must be handled in accordance with the Personal Data Protection Act, No. 9 of 2022 [20].

8.5 Contribution to the Sustainable Development Goals

The Smart Belt contributes mainly to Sustainable Development Goal (SDG) 3, Good Health and Well-being, which aims to ensure healthy lives and promote well-being for all at all ages [23]. By shortening the time between a fall and the arrival of help, it aims to reduce the harm caused by falls in a vulnerable population. By using low-cost components and an open, explainable design, it also supports the wider aim of making assistive health technology accessible to people with limited resources.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the Smart Belt, the waist-worn fall detection component of the SmartGlove healthcare IoT system for people with PD. Falls are frequent in this population, and people who fall may be unable to call for help. The belt addresses this by detecting falls automatically on the device, using an MPU6050 accelerometer filtered in hardware and a two-stage rule in which an impact above 24.0 m/s² must be followed within 1.5 s by a trunk tilt above 55°. A confirmed fall is latched for 15 s and sent immediately to the local hub, where the backend treats it as an emergency and sends an alert with the user's location through Firebase to carers' phones. The belt also reports the wearer's posture, which the system uses as activity context. The design favoured low latency, independence from the network, low power and explainability over the potentially higher accuracy of machine learning. [INSERT: one or two sentences summarising the measured results.]

9.2 Achievement of objectives

Table 9.1 summarises how each specific objective was achieved and the evidence for it.

Table 9.1: Achievement of each specific objective

Objective	Status	Evidence
1. Build the waist-worn sensing unit	Achieved	Section 4.4.1, Figure 4.2, Table 3.1
2. On-device feature extraction	Achieved	Sections 4.4.2 and 5.2.3
3. Two-stage threshold fall detection with latching	Achieved	Sections 4.4.3 and 5.2.4, Figures 4.4 and 4.5
4. Posture and activity classification	Achieved	Sections 4.4.4 and 5.2.5, Figure 4.6
5. Integration with hub and backend	Achieved	Section 5.3, Figure 5.9, Table 6.6
6. Evaluation of detection and alert delivery	Achieved	Tables 6.1 to 6.6

9.3 Individual research and product contributions

My individual research contribution was the design and evaluation of an explainable, on-device two-stage fall detector that combines an impact threshold with a trunk tilt check and is integrated into a multi-sensor monitoring system. My product

contribution was the working Smart Belt prototype, comprising the belt hardware, the complete firmware (belt.ino) for sensing, fall detection, posture classification, local display, hub discovery and telemetry, and the fall handling in the backend that turns a detected fall into an emergency alert with the user's location. The smart glove, the hub's health screening functions, the mobile application and the overall system architecture were developed by the other members of group R26-IT-108.

9.4 Future directions

Several directions would improve the belt. First, a post-fall inactivity check, using the activity variance already computed by the firmware, would align the implementation with the system's definition of a fall and reduce false alarms from people who recover immediately. Second, using the MPU6050's internal sample rate and motion interrupt, or a higher reading rate, would capture short impact peaks more reliably. Third, estimating trunk orientation from all three axes relative to a calibrated upright reference, and fusing the gyroscope, would make the tilt independent of how the belt is worn and of the direction of the fall. Fourth, a short cancellation window with a button would let the wearer cancel false alarms. Fifth, the thresholds should be evaluated on public datasets such as SisFall [17] and, with ethical approval, on data from people with PD; this data could also be used to train a compact machine learning model whose benefit over the threshold rule can then be measured. Finally, the communication should be secured with encrypted and authenticated connections, credentials should be provisioned rather than compiled into the firmware, and the battery life should be measured and optimised.

REFERENCES

[1] B. R. Bloem, Y. A. M. Grimbergen, M. Cramer, M. Willemsen, and A. H. Zwinderman, "Prospective assessment of falls in Parkinson's disease," J. Neurol., vol. 248, no. 11, pp. 950–958, 2001, doi: 10.1007/s004150170047.

[2] R. M. Pickering et al., "A meta-analysis of six prospective studies of falling in Parkinson's disease," Mov. Disord., vol. 22, no. 13, pp. 1892–1900, 2007, doi: 10.1002/mds.21598.

[3] J. Fleming and C. Brayne, "Inability to get up after falling, subsequent time on floor, and summoning help: Prospective cohort study in people over 90," BMJ, vol. 337, Art. no. a2227, 2008, doi: 10.1136/bmj.a2227.

[4] R. Igual, C. Medrano, and I. Plaza, "Challenges, issues and trends in fall detection systems," Biomed. Eng. Online, vol. 12, Art. no. 66, 2013, doi: 10.1186/1475-925X-12-66.

[5] N. Noury et al., "Fall detection – Principles and methods," in Proc. 29th Annu. Int. Conf. IEEE Eng. Med. Biol. Soc. (EMBS), Lyon, France, 2007, pp. 1663–1666, doi: 10.1109/IEMBS.2007.4352627.

[6] M. Kangas, A. Konttila, P. Lindgren, I. Winblad, and T. Jämsä, "Comparison of low-complexity fall detection algorithms for body attached accelerometers," Gait Posture, vol. 28, no. 2, pp. 285–291, 2008, doi: 10.1016/j.gaitpost.2008.01.003.

[7] A. T. Özdemir, "An analysis on sensor locations of the human body for wearable fall detection devices: Principles and practice," Sensors, vol. 16, no. 8, Art. no. 1161, 2016, doi: 10.3390/s16081161.

[8] Espressif Systems, "ESP32-S3 Series Datasheet," Ver. 2.2, Mar. 2026. [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents (accessed Oct. 10, 2026).

[9] InvenSense Inc., "MPU-6000 and MPU-6050 Product Specification," Doc. PS-MPU-6000A-00, Rev. 3.4, 2013.

[10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," IEEE Internet Things J., vol. 3, no. 5, pp. 637–646, Oct. 2016, doi: 10.1109/JIOT.2016.2579198.

[11] F. Bagalà et al., "Evaluation of accelerometer-based fall detection algorithms on real-world falls," PLoS ONE, vol. 7, no. 5, Art. no. e37062, 2012, doi: 10.1371/journal.pone.0037062.

[12] A. L. Silva de Lima et al., "Home-based monitoring of falls using wearable sensors in Parkinson's disease," Mov. Disord., vol. 35, no. 1, pp. 109–115, 2020, doi: 10.1002/mds.27830.

[13] A. T. Özdemir and B. Barshan, "Detecting falls with wearable sensors using machine learning techniques," Sensors, vol. 14, no. 6, pp. 10691–10708, 2014, doi: 10.3390/s140610691.

[14] A. K. Bourke, J. V. O'Brien, and G. M. Lyons, "Evaluation of a threshold-based tri-axial accelerometer fall detection algorithm," Gait Posture, vol. 26, no. 2, pp. 194–199, 2007, doi: 10.1016/j.gaitpost.2006.09.012.

[15] D. M. Karantonis, M. R. Narayanan, M. Mathie, N. H. Lovell, and B. G. Celler, "Implementation of a real-time human movement classifier using a triaxial accelerometer for ambulatory monitoring," IEEE Trans. Inf. Technol. Biomed., vol. 10, no. 1, pp. 156–167, Jan. 2006, doi: 10.1109/TITB.2005.856864.

[16] M. J. Mathie, A. C. F. Coster, N. H. Lovell, and B. G. Celler, "Accelerometry: Providing an integrated, practical method for long-term, ambulatory monitoring of human movement," Physiol. Meas., vol. 25, no. 2, pp. R1–R20, 2004, doi: 10.1088/0967-3334/25/2/R01.

[17] A. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, "SisFall: A fall and movement dataset," Sensors, vol. 17, no. 1, Art. no. 198, 2017, doi: 10.3390/s17010198.

[18] E. Casilari, J.-A. Santoyo-Ramón, and J.-M. Cano-García, "Analysis of public datasets for wearable fall detection systems," Sensors, vol. 17, no. 7, Art. no. 1513, 2017, doi: 10.3390/s17071513.

[19] Apple Inc., "Use Fall Detection with Apple Watch," Apple Support. [Online]. Available: https://support.apple.com/en-us/108896 (accessed Oct. 10, 2026).

[20] Personal Data Protection Act, No. 9 of 2022, Parliament of the Democratic Socialist Republic of Sri Lanka, 2022. [Online]. Available: https://documents.gov.lk/view/acts/2022/3/09-2022_E.pdf

[21] Medical Device Software – Software Life Cycle Processes, IEC 62304:2006+AMD1:2015, International Electrotechnical Commission, 2015.

[22] Medical Devices – Application of Risk Management to Medical Devices, ISO 14971:2019, International Organization for Standardization, 2019.

[23] United Nations, "Goal 3: Ensure healthy lives and promote well-being for all at all ages." [Online]. Available: https://sdgs.un.org/goals/goal3 (accessed Oct. 10, 2026).

GLOSSARY

Activity of daily living – A routine everyday activity such as walking, sitting down or lying down, used in fall detection testing as the negative class.

Alert latching – Keeping an alert active for a fixed period after it is triggered, even if the triggering condition ends.

Impact threshold – The value of the signal magnitude vector above which the belt considers that an impact has occurred.

Local hub – The computer, phone or Raspberry Pi on the local network that receives data from the wearables and runs the backend.

Post-impact window – The period after an impact during which the tilt condition must be met for a fall to be confirmed.

Posture state – The belt's classification of the wearer's current posture or activity.

Tilt angle – The angle between the sensor's x-axis and the horizontal plane, estimated from the direction of gravity.

APPENDICES

Appendix A Smart Belt firmware listing

This appendix lists the complete firmware of the Smart Belt (belt/belt.ino) as used in this work. The Wi-Fi credentials and the Firebase project address have been replaced with <REDACTED> for security reasons.

```
/*
 * Node 2 - Belt Node (Waist / Spine Fall Detection), ESP32-S3 SuperMini
 * File: belt/belt.ino
 * GPIO 8 = shared I2C SDA, GPIO 9 = shared I2C SCL.
 *
 * Telemetry Transport: Method 2 — Local Python AI Backend Hub (/api/telemetry)
 * (The Python Backend computes 5 ML models + Fusion + Clinical Rules and syncs to
Firebase)
 *
 * Optimizations Applied:
 *   [✓] Wire.setClock(400000) — I2C 400 kHz Fast Mode (4x throughput, ~6.5ms OLED push)
 *   [✓] Decoupled OLED refresh at 1 Hz (1000 ms timer) — 95%+ drop in display bus load
 *   [✓] Synchronized WiFi credentials (credentials redacted)
 *   [✓] 10 Hz high-rate MPU6050 sampling with 2 Hz regular telemetry transmission
 *   [✓] Zero-latency instant burst transmission on newly triggered fall events
 *   [✓] 15-second latching fall state (sustained alert for emergency dispatch)
 *   [✓] Non-blocking WiFi reconnect check (sensors never freeze on packet drops)
 */
#include <WiFi.h>
#include <WiFiUdp.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_SSD1306.h>

// ──────────────────── Posture Types & Prototypes ────────
enum Posture { STANDING, SITTING, LYING, WALKING, RUNNING, UNKNOWN_POSTURE };

// Explicit forward declarations (prevents Arduino preprocessor prototype hoisting errors)
Posture classifyPosture(float tilt, float magnitude, float magVariance);
const char* postureToString(Posture p);
void updateOled(Posture posture, float tilt, float magnitude, float magVariance, bool
isFall);
bool isServerConnected();
bool isWifiConnected();
void scanI2CBus();
void connectNetwork();
void checkNetworkNonBlocking();
String discoverBackendServer();
void sendTelemetry(const String& json);

// ──────────────────── Configuration ─────────────────────
const char* WIFI_SSID     = "<REDACTED>";
const char* WIFI_PASSWORD = "<REDACTED>";

// Dynamic Auto-Discovery Configuration
// 1. Fast local UDP broadcast discovery on port 5005 (<5ms, works offline)
// 2. Falls back to Firebase /hub/config.json if UDP is blocked by router
// 3. Falls back to FALLBACK_SERVER_URL if both fail
const char* FALLBACK_SERVER_URL     = "http://192.168.1.100:5000/api/telemetry";
const char* FIREBASE_HUB_CONFIG_URL = "https://<REDACTED>.firebaseio.com/hub/config.json";

String backendServerUrl = "";
int consecutivePostFailures = 0;

// ──────────────────── Pin Definitions ───────────────────
constexpr int SDA_PIN = 8;
constexpr int SCL_PIN = 9;
```

```
// ──────────────────── Timing Constants ──────────────────
constexpr unsigned long SENSOR_SAMPLE_INTERVAL_MS = 100;   // 10 Hz sampling rate
constexpr unsigned long TELEMETRY_INTERVAL_MS      = 500;   // 2 Hz regular telemetry rate
constexpr unsigned long OLED_REFRESH_INTERVAL_MS   = 1000;  // 1 Hz display refresh rate
constexpr unsigned long WIFI_RECONNECT_INTERVAL_MS = 5000;  // 5s non-blocking retry

// ──────────────────── Fall Detection Parameters ─────────
constexpr float IMPACT_THRESHOLD_MS2        = 24.0;  // ~2.45 g shock threshold
constexpr unsigned long POST_IMPACT_WINDOW_MS = 1500; // 1.5 seconds to settle into lying
angle
constexpr float FALL_TILT_THRESHOLD          = 55.0;  // Degrees (> 55° indicates
lying/ground)
constexpr unsigned long FALL_LATCH_MS        = 15000; // 15 seconds sustained emergency
alert

// ──────────────────── Hardware Objects ──────────────────
Adafruit_MPU6050 mpu;
Adafruit_SSD1306 oled(128, 64, &Wire, -1);
bool mpuReady  = false;
bool oledReady = false;

// ──────────────────── State Timers ──────────────────────
unsigned long lastSensorSample     = 0;
unsigned long lastTelemetrySend    = 0;
unsigned long lastOledRefresh      = 0;
unsigned long lastWifiRetry        = 0;
unsigned long lastSuccessfulPostMs = 0;

bool isServerConnected() {
  return (lastSuccessfulPostMs > 0 && (millis() - lastSuccessfulPostMs < 4000));
}

bool isWifiConnected() {
  return (WiFi.status() == WL_CONNECTED);
}

// ──────────────────── Fall State ────────────────────────
unsigned long impactAtMs       = 0;
unsigned long fallDetectedAtMs = 0;
bool fallActive                = false;
bool lastFallState             = false;

// ──────────────────── Posture Classification ────────────
// Circular buffer for magnitude variance (activity detection)
constexpr int MAG_BUFFER_SIZE = 20;  // ~2 seconds window at 10 Hz
float magBuffer[MAG_BUFFER_SIZE];
int magBufferIndex = 0;
bool magBufferFull = false;

float computeMagVariance() {
  int count = magBufferFull ? MAG_BUFFER_SIZE : magBufferIndex;
  if (count < 5) return 0.0f;
  float sum = 0.0f, sumSq = 0.0f;
  for (int i = 0; i < count; i++) {
    sum += magBuffer[i];
    sumSq += magBuffer[i] * magBuffer[i];
  }
  float mean = sum / (float)count;
  return (sumSq / (float)count) - (mean * mean);
}

Posture classifyPosture(float tilt, float magnitude, float magVariance) {
  if (abs(tilt) > 60.0f) return LYING;
  if (magVariance > 12.0f) return RUNNING;
  if (magVariance > 5.0f)  return WALKING;
  if (abs(tilt) < 20.0f)  return STANDING;
  if (abs(tilt) < 45.0f)  return SITTING;
  return STANDING;
}

const char* postureToString(Posture p) {
  switch (p) {
    case STANDING: return "STANDING";
    case SITTING:  return "SITTING";
    case LYING:    return "LYING";
    case WALKING:  return "WALKING";
    case RUNNING:  return "RUNNING";
```

```
    default:       return "UNKNOWN";
  }
}

// ──────────────────── Non-Blocking Network ─────────────
void connectNetwork() {
  if (WiFi.status() == WL_CONNECTED) return;
  Serial.print("[WiFi] Connecting to ");
  Serial.println(WIFI_SSID);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  unsigned long startWait = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - startWait < 5000) {
    delay(200);
    Serial.print(".");
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\n[WiFi] Connected! IP: " + WiFi.localIP().toString());
  } else {
    Serial.println("\n[WiFi] Initial connect timed out; will retry in background.");
  }
}

void checkNetworkNonBlocking() {
  if (WiFi.status() == WL_CONNECTED) return;
  if (millis() - lastWifiRetry < WIFI_RECONNECT_INTERVAL_MS) return;
  lastWifiRetry = millis();

  Serial.println("[WiFi] Reconnecting in background...");
  WiFi.disconnect();
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}

// ──────────────────── Dynamic Hub Discovery ────────────
String discoverBackendServer() {
  Serial.println("\n[Discovery] Searching for SmartGlove Python Hub on local network...");

  // 1. Fast Local UDP Broadcast Probe (port 5005) - 0ms cloud latency, works offline
  WiFiUDP udp;
  if (udp.begin(0)) {
    IPAddress broadcastIp(255, 255, 255, 255);
    const char* probeMsg = "DISCOVER_SMARTGLOVE_HUB";
    for (int retry = 0; retry < 3; retry++) {
      udp.beginPacket(broadcastIp, 5005);
      udp.write((const uint8_t*)probeMsg, strlen(probeMsg));
      udp.endPacket();

      unsigned long startWait = millis();
      while (millis() - startWait < 350) {
        int packetSize = udp.parsePacket();
        if (packetSize) {
          char reply[128];
          int len = udp.read(reply, sizeof(reply) - 1);
          if (len > 0) {
            reply[len] = '\0';
            String msg = String(reply);
            if (msg.startsWith("SMARTGLOVE_HUB_IP:")) {
              int firstColon = msg.indexOf(':');
              int secondColon = msg.indexOf(':', firstColon + 1);
              if (firstColon != -1 && secondColon != -1) {
                String ip = msg.substring(firstColon + 1, secondColon);
                String port = msg.substring(secondColon + 1);
                backendServerUrl = "http://" + ip + ":" + port + "/api/telemetry";
                Serial.println("[Discovery] ✓ Found Python Hub via UDP: " +
backendServerUrl);
                udp.stop();
                return backendServerUrl;
              }
            }
          }
        }
        delay(10);
      }
    }
    udp.stop();
  }
```

```
  // 2. Firebase Cloud Dynamic Hub Discovery (/hub/config.json)
  Serial.println("[Discovery] Querying Firebase /hub/config for dynamic local IP...");
  WiFiClientSecure secureClient;
  secureClient.setInsecure();
  HTTPClient https;
  if (https.begin(secureClient, FIREBASE_HUB_CONFIG_URL)) {
    int httpCode = https.GET();
    if (httpCode == 200) {
      String payload = https.getString();
      int epIdx = payload.indexOf("\"endpoint\":\"");
      if (epIdx != -1) {
        int start = epIdx + 12;
        int end = payload.indexOf("\"", start);
        if (end != -1) {
          backendServerUrl = payload.substring(start, end);
          Serial.println("[Discovery] ✓ Found Python Hub via Firebase config: " +
backendServerUrl);
          https.end();
          return backendServerUrl;
        }
      }
    }
    https.end();
  }

  // 3. Fallback to default configured URL
  backendServerUrl = String(FALLBACK_SERVER_URL);
  Serial.println("[Discovery] ! Using fallback URL: " + backendServerUrl);
  return backendServerUrl;
}

// ──────────────────── Send Telemetry ────────────────────
void sendTelemetry(const String& json) {
  if (WiFi.status() != WL_CONNECTED) return;
  if (backendServerUrl.length() == 0) discoverBackendServer();

  WiFiClient client;
  HTTPClient http;
  if (http.begin(client, backendServerUrl)) {
    http.addHeader("Content-Type", "application/json");
    int httpCode = http.POST(json);
    if (httpCode > 0) {
      consecutivePostFailures = 0;
      lastSuccessfulPostMs = millis();
      Serial.printf("[Backend] Waist Telemetry POST OK, HTTP: %d\n", httpCode);
    } else {
      consecutivePostFailures++;
      Serial.printf("[Backend] Waist Telemetry POST failed: %s\n",
http.errorToString(httpCode).c_str());
      if (consecutivePostFailures >= 3) {
        Serial.println("[Backend] Multiple connection failures, re-discovering server...");
        discoverBackendServer();
        consecutivePostFailures = 0;
      }
    }
    http.end();
  }
}

// ──────────────────── OLED Display (1 Hz) ─────────────
void updateOled(Posture posture, float tilt, float magnitude, float magVariance, bool
isFall) {
  if (!oledReady) return;

  oled.clearDisplay();
  oled.setTextColor(SSD1306_WHITE);

  bool wifiOk = isWifiConnected();
  bool serverOk = isServerConnected();

  if (isFall) {
    // ═══════════════════════════════════════════
    // EMERGENCY FALL ALERT SCREEN
    // ═══════════════════════════════════════════
    // High-contrast inverted emergency banner
    oled.fillRect(0, 0, 128, 12, SSD1306_WHITE);
```

```
    oled.setTextColor(SSD1306_BLACK, SSD1306_WHITE);
    oled.setTextSize(1);
    oled.setCursor(4, 2);
    oled.print("! EMERGENCY ALERT !");

    // Prominent bold FALL ALERT title
    oled.setTextColor(SSD1306_WHITE);
    oled.setTextSize(2);
    oled.setCursor(4, 16);
    oled.print("FALL ALERT");

    // Impact Magnitude & Posture during fall
    oled.setTextSize(1);
    oled.setCursor(0, 36);
    oled.printf("Impact: %4.1f m/s2 (Lying)", magnitude);
    oled.setCursor(0, 48);
    oled.printf("Tilt  : %4.1f deg", tilt);

    // Network & Dispatch Status
    oled.drawLine(0, 56, 127, 56, SSD1306_WHITE);
    oled.setCursor(0, 57);
    oled.printf("WiFi:%-4s Server:%-4s",
                wifiOk ? "OK" : "NO",
                serverOk ? "ALERT" : "RETRY");
  } else {
    // ═══════════════════════════════════════════
    // NORMAL DASHBOARD: WIFI, SERVER, POSTURE & MAGNITUDE
    // ═══════════════════════════════════════════
    // Top Bar: WiFi & Server connectivity status
    oled.setTextSize(1);
    oled.setCursor(0, 0);
    oled.printf("WiFi:%-4s", wifiOk ? "OK" : "NO");
    oled.setCursor(64, 0);
    oled.printf("Server:%-4s", serverOk ? "OK" : "--");
    oled.drawLine(0, 10, 127, 10, SSD1306_WHITE);

    // Prominent Posture Name (Size 2, centered)
    oled.setTextSize(2);
    const char* pStr = postureToString(posture);
    int xOffset = (128 - ((int)strlen(pStr) * 12)) / 2;
    if (xOffset < 0) xOffset = 0;
    oled.setCursor(xOffset, 15);
    oled.print(pStr);

    // Divider line
    oled.drawLine(0, 34, 127, 34, SSD1306_WHITE);

    // Live Acceleration Magnitude
    oled.setTextSize(1);
    oled.setCursor(0, 38);
    oled.printf("Mag : %5.2f m/s2", magnitude);

    // Metrics Footer: Tilt angle & Activity variance
    oled.setCursor(0, 50);
    oled.printf("Tilt: %4.1f*  Act: %4.1f", tilt, magVariance);
  }

  oled.display();
}

// ──────────────────── I2C Bus Scanner ────────────────────
void scanI2CBus() {
  Serial.println("\n[I2C] Scanning I2C bus on GPIO 8 (SDA) and GPIO 9 (SCL)...");
  byte count = 0;

  for (byte address = 1; address < 127; ++address) {
    Wire.beginTransmission(address);
    byte error = Wire.endTransmission();

    if (error == 0) {
      Serial.printf("[I2C] -> Found device at address 0x%02X", address);
      if (address == 0x68) {
        Serial.println(" (MPU-6050 default with AD0=GND) [OK]");
      } else if (address == 0x69) {
        Serial.println(" (MPU-6050 with AD0=HIGH/3.3V) [OK]");
      } else if (address == 0x3C || address == 0x3D) {
        Serial.println(" (SSD1306 OLED Display) [OK]");
```

```
      } else {
        Serial.println(" (Unknown I2C device)");
      }
      count++;
    }
  }

  if (count == 0) {
    Serial.println("[I2C] ⚠ NO I2C DEVICES FOUND! Check wiring on GPIO 8/9.");
  } else {
    Serial.printf("[I2C] Total devices detected: %d\n\n", count);
  }
}

// ──────────────────── SETUP ─────────────────────────
void setup() {
  Serial.begin(115200);
  delay(2000); // Allow USB CDC enumeration on ESP32-S3
  Serial.println("\n=== Belt Node (ESP32-S3 SuperMini) — Starting ===");

  // Initialize I2C with 400 kHz Fast Mode (4x faster display & sensor transfers)
  Wire.begin(SDA_PIN, SCL_PIN);
  Wire.setClock(400000);

  // Scan I2C bus on boot for physical diagnostics
  scanI2CBus();

  // Initialize MPU6050 (try 0x68, fallback to 0x69)
  if (mpu.begin(0x68, &Wire)) {
    mpuReady = true;
    Serial.println("[MPU6050] Initialized at 0x68 SUCCESS (16G, 500°/s, 21Hz BW)");
  } else if (mpu.begin(0x69, &Wire)) {
    mpuReady = true;
    Serial.println("[MPU6050] Initialized at 0x69 SUCCESS (16G, 500°/s, 21Hz BW)");
  } else {
    Serial.println("[MPU6050] Initialization FAILED at both 0x68 and 0x69!");
    mpuReady = false;
  }

  if (mpuReady) {
    mpu.setAccelerometerRange(MPU6050_RANGE_16_G);
    mpu.setGyroRange(MPU6050_RANGE_500_DEG);
    mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
  }

  // Initialize SSD1306 OLED
  if (!oled.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("[SSD1306] OLED Initialization FAILED!");
    oledReady = false;
  } else {
    oledReady = true;
    oled.clearDisplay();
    oled.setTextColor(SSD1306_WHITE);
    oled.setTextSize(1);
    oled.setCursor(16, 20);
    oled.println("Belt Node");
    oled.setCursor(24, 36);
    oled.println("Initializing...");
    oled.display();
    Serial.println("[SSD1306] OLED Initialized SUCCESS");
  }

  memset(magBuffer, 0, sizeof(magBuffer));

  // Network & Hub Setup
  connectNetwork();
  discoverBackendServer();

  Serial.println("=== Setup Complete ===\n");
}

// ──────────────────── MAIN LOOP ──────────────────────
void loop() {
  checkNetworkNonBlocking();

  // High-rate sensor sampling at 10 Hz (every 100 ms)
  unsigned long nowMs = millis();
```

```
  if (nowMs - lastSensorSample < SENSOR_SAMPLE_INTERVAL_MS) return;
  lastSensorSample = nowMs;

  if (!mpuReady) return;

  sensors_event_t a, g, temperature;
  mpu.getEvent(&a, &g, &temperature);

  float magnitude = sqrt(a.acceleration.x * a.acceleration.x +
                         a.acceleration.y * a.acceleration.y +
                         a.acceleration.z * a.acceleration.z);
  float tilt = atan2(a.acceleration.x,
                     sqrt(a.acceleration.y * a.acceleration.y + a.acceleration.z *
a.acceleration.z)) * RAD_TO_DEG;

  // Update magnitude circular buffer for activity variance calculation
  magBuffer[magBufferIndex] = magnitude;
  magBufferIndex = (magBufferIndex + 1) % MAG_BUFFER_SIZE;
  if (magBufferIndex == 0) magBufferFull = true;
  float magVariance = computeMagVariance();

  // Posture classification
  Posture posture = classifyPosture(tilt, magnitude, magVariance);

  // === Enhanced Fall Detection with Latching State ===
  // 1. Detect impact shock
  if (magnitude > IMPACT_THRESHOLD_MS2) {
    impactAtMs = nowMs;
  }

  // 2. Detect fall: impact followed by lying posture (|tilt| > 55°) within 1.5s
  if (impactAtMs && (nowMs - impactAtMs < POST_IMPACT_WINDOW_MS)) {
    if (abs(tilt) > FALL_TILT_THRESHOLD) {
      fallDetectedAtMs = nowMs;
      impactAtMs = 0; // Consumed trigger
      Serial.printf("[WAIST] !!! FALL DETECTED (Mag: %.1f, Tilt: %.1f°) !!!\n", magnitude,
tilt);
    }
  } else if (impactAtMs && (nowMs - impactAtMs >= POST_IMPACT_WINDOW_MS)) {
    impactAtMs = 0; // Expired impact window without settling into lying posture
  }

  // 3. Sustained Latching: maintain fall alert for FALL_LATCH_MS (15 seconds)
  if (fallDetectedAtMs && (nowMs - fallDetectedAtMs < FALL_LATCH_MS)) {
    fallActive = true;
  } else {
    fallActive = false;
    fallDetectedAtMs = 0;
  }

  // === Telemetry Transmission ===
  // Normal rate: 2 Hz (every 500 ms)
  // Instant burst: newly triggered fall alert or high impact shock (>20 m/s²)
  bool regularDue = (nowMs - lastTelemetrySend >= TELEMETRY_INTERVAL_MS);
  bool instantBurst = (fallActive && !lastFallState) || (magnitude > 20.0f && nowMs -
lastTelemetrySend >= 100);
  lastFallState = fallActive;

  if (regularDue || instantBurst) {
    lastTelemetrySend = nowMs;

    String json;
    json.reserve(320);
    json = "{\"device_id\":\"WAIST_NODE\",\"node_type\":\"waist\",";
    json += "\"timestamp\":" + String(nowMs / 1000.0, 3) + ",";
    json += "\"ax\":" + String(a.acceleration.x, 4) + ",\"ay\":" + String(a.acceleration.y,
4) + ",\"az\":" + String(a.acceleration.z, 4) + ",";
    json += "\"gx\":" + String(g.gyro.x * RAD_TO_DEG, 4) + ",\"gy\":" + String(g.gyro.y *
RAD_TO_DEG, 4) + ",\"gz\":" + String(g.gyro.z * RAD_TO_DEG, 4) + ",";
    json += "\"tilt_degrees\":" + String(tilt, 2) + ",";
    json += "\"posture\":\"" + String(postureToString(posture)) + "\",";
    json += "\"activity_magnitude\":" + String(magVariance, 4) + ",";
    json += "\"fall_detected\":" + String(fallActive ? "true" : "false") + "}";

    sendTelemetry(json);
  }
```

```
  // === Decoupled OLED Refresh at 1 Hz ===
  if (nowMs - lastOledRefresh >= OLED_REFRESH_INTERVAL_MS) {
    lastOledRefresh = nowMs;
    updateOled(posture, tilt, magnitude, magVariance, fallActive);
  }
}
```

Appendix B Academic integrity and AI use disclosure

This appendix discloses the use of generative AI tools in preparing this thesis, in accordance with the module policy. Table B.1 lists each tool, the purpose of its use, the sections or artefacts affected and the verification performed by the student. The content of the thesis is based on the student's own project files, and the student remains responsible for its accuracy.

Table B.1: Use of generative AI tools

Tool	Purpose	Sections or artefacts affected	Verification performed by the student
Claude (Anthropic), conversational AI assistant	Reviewing the project files and identifying inconsistencies and open questions	Planning; open questions on all chapters	I reviewed each flagged issue, checked it against my own code, test records and thesis text, and corrected or removed every item I judged to be incorrect.
Claude (Anthropic)	Proposing the thesis structure, objectives and research questions for the student's approval	Chapter 1; document structure	I reviewed the proposed structure and objectives, revised them to match my actual implementation, and confirmed the final wording with my supervisor.
Claude (Anthropic)	Searching for and checking literature references	Chapter 2; Reference List	I independently located and read each cited source, verified the bibliographic details and the claims attributed to it, and removed or corrected references that did not support the text.
Claude (Anthropic)	Drafting text from the student's code and project documents	Chapters 1-9; Abstract; Glossary	I checked every drafted paragraph against my firmware (belt.ino), backend code (main.py) and test records, and rewrote sections where the draft did not match the implementation.

Tool	Purpose	Sections or artefacts affected	Verification performed by the student
Claude (Anthropic)	Generating design diagrams from belt.ino	Figures 1.1, 2.1-2.3, 3.1, 4.1, 4.2, 4.4-4.6, 5.9	I verified each diagram against the firmware logic and the architecture, corrected the axis labels, decision branches and connections, and confirmed the final versions.
Claude (Anthropic)	Formatting the thesis into the SLIIT template	Whole document	I checked the formatting, headings, figure and table numbering, and references against the SLIIT template and corrected any remaining errors.
Claude (Anthropic)	Cross-checking supplied files against the thesis and correcting wording in a test case and a limitation	Sections 3.7, 5.1, 5.4.1, 7.5.3; Table 5.2	I confirmed each correction against the source files, accepted the changes I agreed with, and made further edits to the test case and the limitation to reflect my actual results.